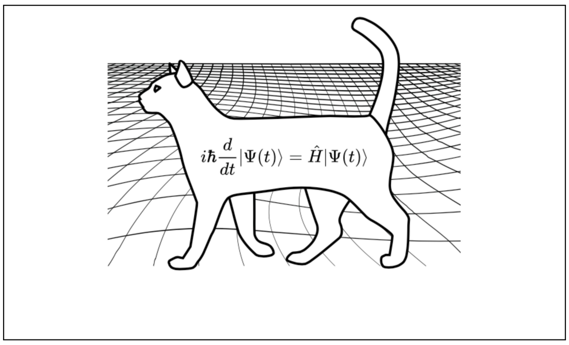
Thesis for the degree in Computer Engineering

Schrödinger Memory

A memory game with quantum computing and entanglement

David José Giovannini

2026 - Epoch 1767225600 Time - 5787


$i\hbar\frac{d}{dt}|\Psi(t)\rangle = \hat{H}|\Psi(t)\rangle$

Universitas Mercatorum
Faculty L8 Computer Engineering
Telematic University of the *Camere di Commercio Italiane*

Dean

Prof. Giovanni Cannata

Referees

Prof.ssa Barbara Martini, *director of the Department of Engineering and Science*

Prof. Luca Potì, *head of research area at National Consortium for Telecommunications*

Date of the graduation

04.11.2026

To my mother and my father, for the most important lesson of life.

“La parte migliore di noi vive in eterno.”

Contents

Abstract 1

1. Math behind the game 3
- 1.1. Overview 3
- 1.2. Differences between bit and qubit 6
- 1.3. Probabilistic or deterministic in quantum states 8
- 1.4. Quantum teleport and entanglement 9

2. Quantum entanglement and memory game 15
- 2.1. Overview 15
- 2.2. The rules of memory game 16
- 2.3. Quantum circuits for deck generation 17
- 2.4. Quantum circuits for cards visualization 19
  - 2.4.1. Experiment 1: absence of entanglement 19
  - 2.4.2. Experiment 2: partial and variable entanglement 20
  - 2.4.3. Experiment 3: the Bell state basis 20
  - 2.4.4. Experiment 4: entanglement grows with pairs found 22
  - 2.4.5. Experiment 5: entanglement not manifest in measurement probabilities 23

3. Qubit Visualization 25
- 3.1. Overview 25
- 3.2. Reading graphs: complex numbers, vectors, and the Bloch sphere 26
  - 3.2.1. Complex numbers as vectors 26
  - 3.2.2. Describing a vector by its numbers 27
  - 3.2.3. Vector length and probability 27
  - 3.2.4. The Bloch sphere 28
- 3.3. Play and learn 29
  - 3.3.1. Experiment 1: two independent qubits 29
  - 3.3.2. Experiment 2: partially entangled vectors 31
  - 3.3.3. Experiment 3: Bell states and density matrix 32
  - 3.3.4. Experiment 4: progressive increase of entanglement 34
  - 3.3.5. Experiment 5: maximally entangled state exhibiting 36

4. Experiments on a real quantum computer 39
- 4.1. Overview 39
- 4.2. The quantum processor unit or QPU 39

4.3. Real runnings = real results . . . 40
4.3.1. Experiment 1: two independent qubits . . . 41
4.3.2. Experiment 2: partially entangled vectors . . . 42
4.3.3. Experiment 3: Bell states and density matrix . . . 44
4.3.4. Experiment 4: progressive increase of entanglement . . . 46
4.3.5. Experiment 5: maximally entangled state exhibiting . . . 47

5. Software and architecture 49
5.1. Overview . . . 49
5.2. Initialization and deployment . . . 49
5.3. Software architecture . . . 50
5.3.1. Configuration layer . . . 51
5.3.2. Quantum backend layer . . . 51
5.3.3. Visualization layer . . . 52
5.3.4. Game and interface layer . . . 52
5.3.5. Software design choices . . . 53

6. Conclusion 59
6.1. Overview . . . 59
6.2. From simulation to real hardware . . . 59
6.3. Practical limits of access to quantum hardware . . . 60
6.4. The use of artificial intelligence . . . 61
6.5. What this work has meant for learning . . . 62
6.6. Limitations and future work . . . 62
6.7. Final remarks . . . 63

Acknowledgments 65

A. Appendix 67
A.1. Overview . . . 67
A.2. Links . . . 67
A.3. License . . . 68
A.4. How to run the experiments . . . 68
A.5. Images and file names . . . 68
A.6. Dates of the hardware runs . . . 69

Bibliography 71

Abstract

This thesis it's my personal way, to learn **quantum computing** and all the potential behind it. Practice has helped me always a lot to discover dynamics that sometimes are difficult to understand. Writing source code with the objective of learn something complex like quantum computing is a good interactive way that I hope will let me discover something new about what I'm thinking about *reality.*

For this reason I have decided to create a *memory game* that can show some interesting properties of quantum computing, especially for the **entanglement** state. The source code that I have created in **Python** with the **Qiskit** module, will be tested on a real quantum computer to compare my analysis and experience with a *real* use case.

The name "*Schrödinger Memory*" has been chosen because one of the most important experiment designed by the physicist **Erwin Schrödinger** was to image a cat that was in an uncertain state between life and death[1].

Since I love cats, and I don't want to hurt any animal for this experiments; I have thought that the creation of a game to prove the **superposition** of quantum bits could be a good exercise to better learn how is moving the reality of quantum computing.

The proprieties that are expressed from quantum computing are really different from the classical physic used by modern computers based on semiconductor and electromagnetism, my experiments in this thesis are addressed to better understand something never tested before on *classical computers* like the **entanglement** state, with two **qubits**.

The development of this game give me also, the possibility to improve my programming skills with the creation of a source code that respect my vision about *freedom in technology.*

To realize the source code I have used only **free** and **open source software**, because the technology that we are creating have to consider *ethical* and *philosophical* values that grant everyone the utilization, analysis and modification of the code realized with this experiments; like the *four freedoms* of **FOSS**:

Freedom 0: The freedom to run the program as you wish.
Freedom 1: The freedom to study how the program works.
Freedom 2: The freedom to redistribute copies so you can help others.
Freedom 3: The freedom to distribute copies of your modified versions to others.

[2, Philosophy of Free Software]

The new paradigm of using computers include the **artificial intelligence** (later in the text **AI**), and for this reason I have used some AIs in *oversight mode*, because it is the best way to solve problems avoiding humans and machines errors; this way to work gave me the ability to speedup the development of source code, granting me more time to: read new articles, follow quantum courses or fixing **bgus**.

Before starting with my thesis, I would like to apologize for any inaccuracies or omissions that may appear in this work.

With this thesis, I simply attempt to write down, with humbleness, the method I applied to approach studying quantum computers.

I apologize once more, for the inaccuracy that may appear in this text.

1. Math behind the game

1.1. Overview

The experiments that I have realized is using some fundamentals notions of math that must be understood to learn how the quantum computers are working. The first concept that we have to familiarize with is the *quantum bit* or **qubit**.

The classical computers are using **bits** that could be explained as the state assumed by a switch when we want to turn *ON* or *OFF* the light. This simple action that we do daily, explain better the state that can assume a *single bit*. Thinking with numbers, a single bit can be expressed in terms of electrical tension like *+0V* or *+5V*, needed to give power to the bulb; but moving to computers the standard way to represent a single bit use the values 0 or 1 that it means presence or absence of voltage.

This kind of data, give us the ability to express simple information like the presence or not of electrical tension in the case of the switch; or **true** and **false** if we think to the **boolean logic**.

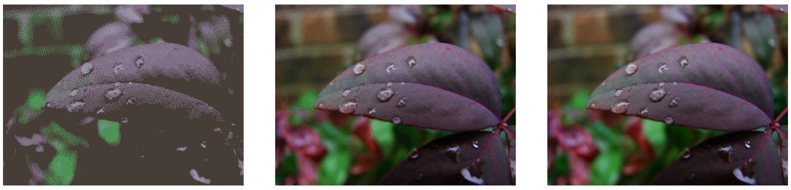
The computers of current times are using bits elevated at base two, to make everything; from complex calculations to storing data. To make it possible we are using several number of bits to trying to describe the *real world*.

For instance, with 2^8 bits we can represent 256 values as better described in the table below:

Decimal	**8-bit binary**
0	0000 0000
1	0000 0001
...	...
254	1111 1110
255	1111 1111

This 256 combinations associated to colors could represent the shapes of gray of the old photos in black and white, with an oblivious limitation in term of the reality representation like we are commonly intended to see it. The **quantization** that we are applying to the colors of a picture can give us an idea on how many data we have to process with computers.

Below you can see the same pictures with different color depth:



2 bit 8 bit 24 bit

With the relatively low color depth, the stored value is typically a number representing the index into a color map or palette (a form of vector quantization). The colors available in the palette itself may be fixed by the hardware or modifiable by software. Modifiable palettes are sometimes referred to as pseudocolor palettes.

[3, 4, 5, 6, Same image on three different color depths]

If you are not really aware about how many data are needed to represent a photo with colors, try to think that a common phone camera with 12 megapixel and 24 bit (8 bit for each color RGB) in RAW format (without any compression) can store a relatively big amount of data.

Usually humans are reasoning using numbers with a scale of 10s till a maximum of 1000s, but in the computer world the impact of a simple file have an amplitude of millions or billions of numbers.

Here some calculations to see approximately how many data storage we need for a *simple photo* in term of bits and mega bytes:

$$\textbf{Total Bit} = \text{Spatial Sampling} \times \text{Color Quantization}$$

$$\textbf{Total Bit} = 12\ 000\ 000 \text{ pixel} \times 24 \text{ bit} = 288\,000\,000\,\text{bit} = 36\,\text{MB}$$

The CPU architectures nowadays are processing data in words of 64 bits, so a single word can represent 2^{64} different values, and can store it in registers of up to 128 bits, which can represent up to 2^{128} different values, for the common general purpose **x86_64** computers architecture.

The fundamental data types are bytes, words, doublewords, quadwords, and double quadwords. A byte is eight bits, a word is 2 bytes (16 bits), a doubleword is 4 bytes (32 bits), a quadword is 8 bytes (64 bits), and a double quadword is 16 bytes (128 bits).

[7, Intel 64 and IA-32 Architectures Software Developer's Manual]

Also if this numbers are out the imagination of a common person; the reality, the nature, the engineering and the science needs enormous computing power to process data to give simple answers to humans like, the famous comedy science fiction *The Hitchhiker's Guide to the Galaxy*:

«O Deep Thought computer," he said, "the task we have designed you to perform is this.
*We want you to tell us...." he paused, "**The Answer.**"*
"The Answer?" said Deep Thought. "The Answer to what?"
*"**Life!**" urged Fook.*
*"**The Universe!**" said Lunkwill.*
*"**Everything!**" they said in chorus.*
Deep Thought paused for a moment's reflection.
"Tricky," he said finally.
"But can you do it?"
Again, a significant pause.
"Yes," said Deep Thought, "I can do it."
"There is an answer?" said Fook with breathless excitement.
*"Yes," said Deep Thought. "**Life, the Universe, and Everything.***
There is an answer.
But, I'll have to think about it."
[...]
Fook glanced impatiently at his watch.
"How long?" he said.
"Seven and a half million years," said Deep Thought.
[...]
*"**Forty-two,**" said Deep Thought, with infinite majesty and calm.»*
[8, Douglas Adams, The Hitchhiker's Guide to the Galaxy]

The quantum computing seems give us access to a new scale of magnitudes of computational power, that could grant us the solutions for more complex kind of problems that current CPU or GPU are not able to do.

For instance a Large Scale Problem (**LSP**) that now we are not able to solve, is to find the optimized travel length that Santa Claus [9] do to deliver all the gifts for Christmas; this enigma is a variation of the *Traveling Salesman Problem* (later in the text is mentioned as **TSP**) that is a classical operative research optimization problem.

TSP is not limited to physical distances, but any other optimization function in place of the distance can be used. In the early days, travel distance or travel time were used (Dantzig and

Ramser, 1959), but currently, other objectives also exist, such as minimizing the exposure to sunlight (Li et al., 2019), maximizing the safety of travel (Krumm and Horvitz, 2017), or minimizing CO2 levels (Kuo and Wang, 2011; Lin et al., 2014). Practical applications include logistics such as vacation planning or orienteering (Fränti et al., 2017), transportation of goods, providing maintenance, or offering health services (Golden et al., 2008), among others.

[10, Solving the Large-Scale TSP Problem in 1 h: Santa Claus Challenge 2020]

With the qubit the new era of quantum computation will give to the human beings the ability to solve new challenges in the fields of: math, science, medicine and knowledge of the universe.

Representing a generational advancement in computer science, QPUs are designed to process quantum algorithms better than even the most powerful supercomputers. Optimized for large-scale quantum calculations, QPUs are not intended to replace CPUs. Instead, QPUs are being integrated into high-performance computing (HPC) systems alongside CPUs and graphics processing units (GPUs).

In a quantum-centric supercomputer, each type of processor functions differently and is used for processing different types of computations with the ecosystem:

- ***CPUs**: Central processing units (CPUs) process inputs sequentially, performing tasks in a linear fashion and are best suited for high-level control operations, such as managing data across different system components.*
- ***GPUs**: Graphics processing units (GPUs) excel at parallel processing large amounts of operations simultaneously. GPUs can potentially be used in quantum systems to offload some amount of the processing workload from QPUs.*
- ***NPUs**: Neural processing units (NPUs) mimic the function of the human brain and are typically used for tasks involving the specific types of computations used in machine learning (ML) and artificial intelligence (AI).*
- ***QPUs**: Quantum processing units (QPUs) process information by using qubits instead of binary bits and are designed to perform complex quantum algorithms. QPUs are best used for certain kinds of highly complicated problems, and many of today's promising quantum algorithms provide probabilistic solutions instead of precise answers.*

[11, QPU (Quantum Processing Unit)]

1.2. Differences between bit and qubit

In this thesis we will not cover the physical aspects behind bit and qubit, it is out of the scope of a computer engineering degree. For a computer engineer is more interesting to learn how many data is possible to store with a single qubit.

To do that, let's do a bet! I know that this can sound weird, but it's not a joke! Let's go back to the concept of binary value. As we said, a bit can have 2 simple state; usually we think to 0 or 1.

Now let's imagine, that instead having a bit; we have a coin with two different sides: *head* or *tail*. If we flip the coin 10 times we can obtain one side or another of the coin. The percentage of times that we can obtain head or tail can differ as this value is random.

But increasing the number of repetitions, for instance to 100, we can observe that the probability to have head or tail is equal to 0.5 and that our results will be approximated to a **normal curve** (or **Gaussian distribution**).

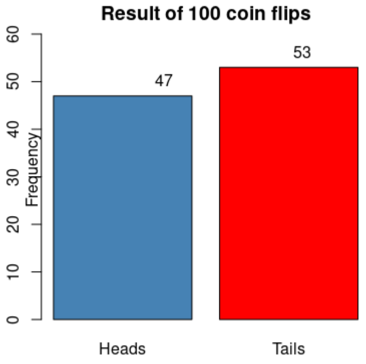


Figure 1.1

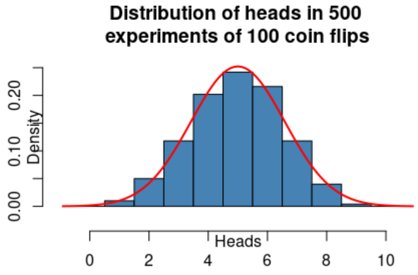


Figure 1.2

Figure 1.1.: *100 repetitions of head tail*

Figure 1.2.: *Normal curve in head tail distribution*

In quantum computers the concept of head or tail is a little bit different. Going back to the famous experiment of Schrödinger's cat we have to think that a coin will be in both states till we don't look it the palm of our hand. It could sound strange, because in our daily life we don't encounter similarities.

Ideally we can think that while the coin is flipping in the air the coin can assume both states and when the coin is catch and seen by a person the state change definitely in head or tail; a quantum bit can exist simultaneously in multiple possible states, and only when measurement is performed it define exactly the state of the bit.

This principle is called **superposition** and could be expressed mathematically with this formula:

$$| 0\rangle \qquad \text{(interpreted as “head”)} \qquad | 1\rangle \qquad \text{(interpreted as “tail”)}$$

$$| \psi\rangle = \frac{1}{\sqrt{2}}(| 0\rangle + | 1\rangle)$$

The meaning of this equation, named **“plus state”** written following the **Dirac notation**; explain better the concept of superposition in a qubit. The qubit can be 0 or 1 with a probability of 50% each. Moving the numbers we can obtain an equation in a easier way to understand:

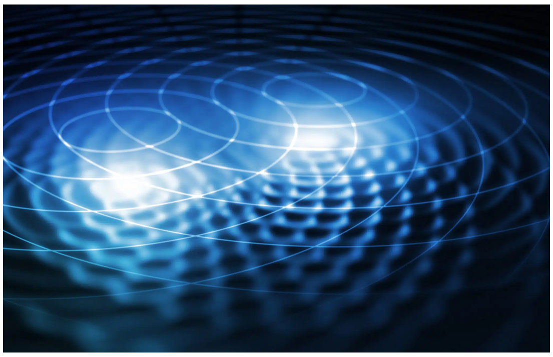
$$P(0) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} \qquad P(1) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

Now it’s clear that the probability (P) that a qubit have to be 0 or 1 is equal to 0.5 and it’s sum is equal to 1. Behind this formula there are other aspects that are going deeper into the quantum mechanics and that I will not consider for this thesis. For my purpose the need is simply to understand the behaviors that a qubit can assume during the computing phase.

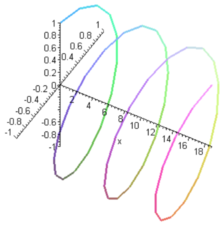
The assumption that a qubit can be both states at the same time bring us again to the example of the coin; the superposition can be imagined as a coin toss; while the coin is flipping into the air and we don’t know if it is head or tail.

1.3. Probabilistic or deterministic in quantum states

The propriety of superposition is something real that we can observe in our daily life, for instance if we throw a stone into a pond, we can see waves form on the surface; if a few moments later we throw another stone, we will see that the waves already existing will be superimposed by the waves formed by the second stone thrown.



[12] Stones in a pond



[13] Wave function

[12, Image form, Schrödinger's Wave Equation]

[13, Animations of solutions to the Schrodinger equation]

If we are at a concert and we are listening music, the same phenomenon happen; also if in this case we are not able to see it with our eyes.

But computers need to process precise data and we can't reason per abstractions, in computers programming we need fix and **reproducible results;** the probability assumed by a qubit must be something repeatable in time without any doubt.

The physic behind quantum computing use wave forms that give us the probability to have a qubit in a certain state, but when we made a measurements of the qubit the state collapse and we obtain as outcome of a measurement a random number, with the probabilities given by the squared amplitudes. What is deterministic is the evolution between measurements.

It is often stated that, unlike classical physics, Quantum Physics is not deterministic. This statement is not really correct, but we need to define carefully what can and what cannot be determined. Very simply, this means that we can only determine the properties of a system that the system actually possesses (pretty obvious really!). In classical mechanics, given an initial condition, defined by the exact position and momentum of a particle at some time, Newton's laws of motion provide us with a set of equations, whose solution will determine the position and momentum of that particle at any subsequent time. The reason why this determination is not possible in Quantum Physics is simply because Heisenberg's Uncertainty relation prevents us from realizing the required initial condition. Remember that this uncertainty relation tells us that the particle does not possess well-defined position and momentum, so it is meaningless to ask whether we can determine them.

On the other hand, if a system is known to be in a certain quantum state, described by a wavefunction at time t1, $\Psi(x, t1)$*, then the solution to Schrödinger's equation gives us exactly the wavefunction,* $\Psi(x, t2)$*, at a later time t2 – and therefore (provided we can solve Schrödinger's equation) we know the quantum state of the system at any later time.*

[14, I Don't Understand Quantum Physics, Douglas Ross FRS]

1.4. Quantum teleport and entanglement

Quantum computers open the way to another field of research that promise to change completely the world that we know; in different fields of computers technology. Since the early days of modernization, the **sci-fi** novels and human imaginary was dreaming a way of moving matter so fast that instantaneously can move the mass of an object, from a place A to a place B.

This idea has been extensively developed in our imagination by television series like **Star Trek**, where the crew of an interstellar ship was using, a so called **transporter** to land explorers on unknown worlds. In the TV series, the idea behind this science fiction machine was that each person's body would be scanned molecule by molecule, then teleport-ed and re-assembled in another location.

It is too optimistic to think that in the next decade we could have instruments that can move matter almost instantaneously, but with the quantum technology we can do some steps forward at lead with telecommunications if we will be able to develop instruments that will use the entanglement state with a new generations of computers.

[15, McCoy, Kirk and Spock in the transporter room]

In August 2008, physicist Michio Kaku predicted in Discovery Channel Magazine that a teleportation device similar to those in Star Trek would be invented within 100 years.[9] Physics students at University of Leicester calculated that to "beam up" just the genetic information of a single human cell (not the positions of the atoms, just the gene sequences) together with a "brain state" would take 4,850 trillion years assuming a 30 gigahertz microwave bandwidth.[10]

[15, Transporter (Star Trek)]

By significantly scaling down the concept of teleportation today, we can see on smaller scale how the entanglement property of qubits allows the *correlations* between the measurement results of entangled qubits appear instantaneously, *but no information travels faster than light* (the *no-communication theorem [16]*).

It turns out that there does in fact exist a form of teleportation that occurs in nature despite the numerous technical roadblocks described in the previous section. It is called quan-

> *tum teleportation, which is based on the well-known concept of quantum entanglement. Erwin Schrödinger coined the word "entanglement" in 1935 in a three-part paper (Schrödinger, 1935a, b, c, 1980). These papers were prompted by the Einstein, Podolsky and Rosen (1935; denoted hereafter as EPR) paper that raised fundamental questions about quantum mechanics, whereby Einstein had loudly complained that quantum mechanics allowed physical processes resembling "spooky action at a distance" to occur. EPR recognized that quantum theory allows certain correlations to exist between two physically distant parts of a quantum system. Such correlations make it possible to predict the result of a measurement on one part of a system by looking at the distant part.*
>
> [17, Teleportation Physics Study]

We are certainly a long way far from being able to teleport organic and non-organic matter; so for now, Captain Kirk's "***energize***" command will remain a nostalgic memory from the past. However, entanglement certainly allows us to: *«[...] boldly go where no man has gone before.»*

Back to reality, one way we can experiment with entanglement state and quantum teleportation is to recreate the connection between two qubits using Qiskit.

Qiskit is a software development kit (later **SDK**), available through Python modules, that allows us to build quantum circuits and algorithms on real quantum computers (typically IBM) or through emulation on your local machine.

To experiment with the entanglement of two qubits, I used the official Qiskit documentation and, as recommended by the developer team, took the course "*Understanding Quantum Information & Computation With John Watrous. [18]*"

All the lessons are enriched with interactive files that can be downloaded and edited using **Jupyter Notebooks**.

I paid particular attention to the lessons on "*Multiple Qubits and Entangled States*" [19], which provide a detailed overview from both a mathematical and programming perspective.

The interactive code available online and written with Jupiter Notebook, is very helpful in research and study; for instance, other experiments could be made increasing the number of qubits to be connected directly from your **browser**.

However, to avoid adding further complexity to a topic that I'm still trying to explore, I have focused my attention to the entanglement of two qubits; or more commonly called the **Bell state** [20] from the physicist **John Stewart Bell** [21]. Bell did not receive the Nobel Prize, but the 2022 Nobel Prize in Physics was awarded to Alain Aspect, John Clauser and Anton Zeilinger for experiments on Bell's theorem.

Following the instructions provided by Qiskit documentation is quite easy to have a snippet to create a quantum circuit with quantum entanglement. To do that is necessary to familiarize with the basics quantum **logical gates** to build an **algorithm**.

To be more detailed the entanglement states between two qubits could be expressed by four equations, written as follow:

$$| \phi^{+}\rangle = \frac{|00\rangle+|11\rangle}{\sqrt{2}}$$

$$| \phi^{-}\rangle = \frac{|00\rangle-|11\rangle}{\sqrt{2}}$$

$$| \psi^{+}\rangle = \frac{|01\rangle+|10\rangle}{\sqrt{2}}$$

$$| \psi^{-}\rangle = \frac{|01\rangle-|10\rangle}{\sqrt{2}}$$

The most easiest state $| \phi^{+}\rangle$ in python code could be translated as:

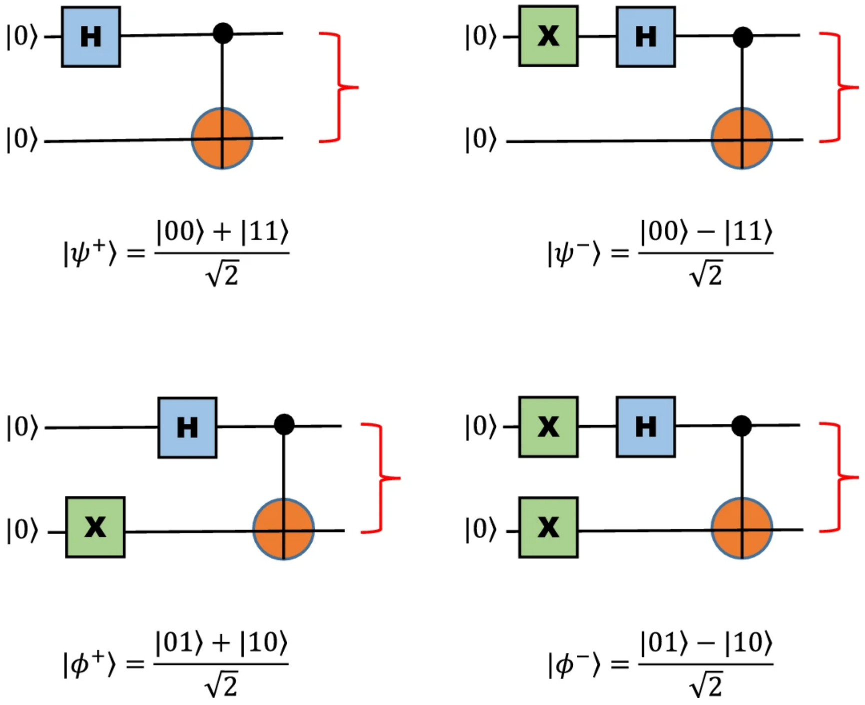
1. two qubits both starting in $| 0\rangle$
2. Hadamard on qubit 0 → puts it in superposition
3. CNOT (control = qubit 0, target = qubit 1) → links them together

Later in the next Chapter 2 on page 17 we will see in details all the quantum circuits used in the source code, but for now the goal is to show how simply is to write quantum circuits with Qiskit. With exactly four lines of Python code we can translate the mathematical equation $| \phi^{+}\rangle$ for our simulations:

```
qc = QuantumCircuit(2)
# Apply H-gate to the first:
qc.h(0)
# Apply a CNOT:
qc.cx(0,1)
qc.draw()
```

[19, Source code from paragraph, 3.1; The CNOT-Gate]

Below you can see the quantum circuits graphical representation, for all the four states:


$|0\rangle$
H
$|0\rangle$
$|0\rangle$
X
H
$|0\rangle$
$|\psi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$
$|\psi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}$
$|0\rangle$
H
$|0\rangle$
X
$|0\rangle$
X
H
$|0\rangle$
X
$|\phi^+\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}}$
$|\phi^-\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}}$

[22, Quantum circuits for Bell state]

Once realized the entanglement state we are able to use all the properties expressed before, the correlations between two qubits is called **ebit** (or *entanglement bit*); it is the fundamental unit of quantum entanglement, a measure of how much entanglement two parts share.

In the next Chapter on page 15 we will see how all these concepts will be used to create an educational and interactive game in which the concept of fun is oriented towards learning through programming.

2. Quantum entanglement and memory game

2.1. Overview

A good way to improve my development skills and learn new concepts in a fun way, has been to write a simple game in Python to test the capabilities of quantum computers. To develop something interesting and educational, I thought that a simple game could have shown, in an interactive way, the concept of entanglement.

Thinking more about the particularities of quantum entanglement I have found that a nice example to show the aspects of *connections* between qubits, could have been represented by the *memory game*.

[23, Memory Ravensburger]

A traditional common game that a lot of people have experienced in the childhood and that improve the memory of the players, challenging to remember the biggest number of couples of cards with the same images; in my case the game has been revisited with a touch of quantum computing to better understand the properties behind this new technology.

The experiments that I have done, with the quantum memory game; are showing several versions of entanglement. Using a graphical user interface (later **GUI**) written in **Qt 6** I have realized 16 *virtual cards* with 8 couples of **emojis** that are *virtually associated* to a qubit.

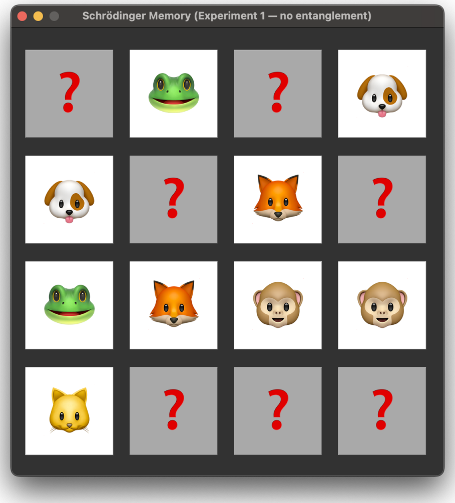


Figure 2.1.: *image of the GUI common to all experiments*

2.2. The rules of memory game

The *memory game*, also known as *concentration*, is a simple card-matching game played by one or more players. To set up, all cards are laid face down on the table in a grid. Each card has a matching twin, so the deck contains pairs of identical images.

On each turn, a player flips over two cards of their choice, one at a time. If the two cards match, the player keeps the pair, removes them from the table, and earns another turn. If they do not match, the cards are turned back face down in their original positions, and play passes to the next player.

The key skill is memory: players must remember the position of the cards they have seen before, even after they are flipped back over, so they can later match them.

The game continues until all pairs have been collected. The winner is the player with the most pairs at the end, or the player who collects all pairs when playing alone.

In this version of the game, the player that will find the equal card will visualize a pop up image that will explain the quantum state of the ebit; to be more educational and to investigate deeply on the quantum entanglement I have created 5 versions of the game as experiments to run in a first phase as simulation and then to run it on a real quantum computer.

Here you can find all the Python files that you will find in the **Git** repository (follow this link to access the Shrodinger Memory source code) for this thesis; each file is corresponding to a different experiment on quantum computing:

- **Experiment 1:** No entanglement, opposite arrows
  (filename: `exp1_no_entanglement_random_qubits.py`)
- **Experiment 2:** Random partial entanglement per pair
  (filename: `exp2_bell_concurrence.py`)
- **Experiment 3:** All four Bell states
  (filename: `exp3_all_bell_states.py`)
- **Experiment 4:** Entanglement that grows with progress
  (filename: `exp4_tunable_entanglement.py`)
- **Experiment 5:** Phase-type entanglement, different gate family
  (filename: `exp5_phase_equatorial.py`)

2.3. Quantum circuits for deck generation

A quantum circuit is a visual and mathematical representation of a quantum computation, analogous to an electronic circuit in classical computing. It consists of a sequence of quantum gates, measurements, and sometimes resets, applied to a set of qubits over time.

The qubits are usually depicted as horizontal lines, each representing one two level quantum system. Time flows from left to right. A gate acting on a qubit is shown as a box or symbol placed on the qubit's line at the appropriate time; multi-qubit gates (like CNOT or SWAP) connect several lines. The gates implement unitary transformations on the state of the qubits. Measurement gates, often shown as a meter symbol, collapse the quantum state to a classical outcome 0 or 1.

Quantum circuits are built from a small universal set of gates, e.g., **Hadamard** (creates superposition), **Pauli** X, Y, Z (bit and phase flips), and **CNOT** (creates entanglement). By composing these, any unitary operation on an arbitrary number of qubits can be approximated (the Solovay-Kitaev theorem [24]).

When a circuit is executed on a quantum computer, the initial state (usually $|0...0\rangle$) is transformed by each gate in sequence, and finally the qubits are measured, producing a string of classical bits. Because of superposition and entanglement, the output of a circuit is generally probabilistic; multiple runs (shots) are taken to estimate the distribution of results.

The circuits created for the experiments have a common schema but they differs to change the behaviors in the game. The main circuit used for all the games could be graphically represented as the following image:

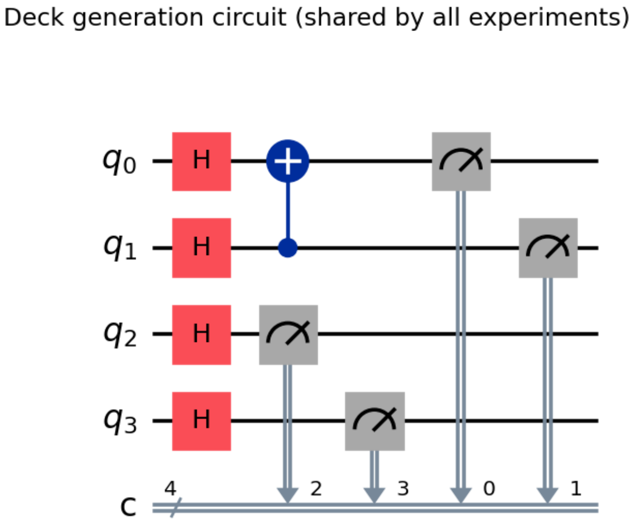


Figure 2.2.: *common circuit to all the experiments*

In the image you can see 4 qubits (q0, q1, q2, q3), each put into equal to $|0\rangle$ at the starting position. The H (Hadamard) gate on each qubit means that, before measurement, every one of the 4 qubits has a 50/50 chance of coming out 0 or 1 and this is the source of the *quantum randomness* that I have tried to add to the whole game to generate more interesting visualizations.

One entangling gate cx(1, 0), represented in the image as a blue circle with a plus in the middle that connect the q0 to q1, is a CNOT. The qubit 1 acts as the controller and qubit 0 as the target. If qubit 1 measures 1, qubit 0 value gets flipped; if qubit 1 measures 0, qubit 0 is left alone.

All 4 qubits measured simultaneously, giving a random 4 bit string used to build one card:

- qubit 0 bit → give a the sign to the card, positive or negative (+ or -)
- qubits 1, 2, 3 bits → combined into a 3 bit number (0 - 7) → picks which emoji out of the 8

This whole circuit runs once per card, discarding duplicates, until all 16 distinct sign and emoji combinations exist.

After the deck generation, the last thing that happens in the circuit is the *measurement*. This operation is destructive in quantum mechanics; the instant a qubit is measured, its superposition collapses into one definite classical value (0 or 1), and that quantum information is lost. After that the Python code consider only a plain 4 bit string (like 0011); not a quantum state anymore but only four classical bits.

So during the visualization of the popup after match, the source code builds a fresh 2 qubit circuit from scratch, starting clean at $|00\rangle$, and uses the matched card's sign and emoji identity only as classical inputs that decide which gates to apply depending to the experiment and not as some inherited quantum state. That's what makes it possible to have a controlled, reproducible, exactly known state to actually show you: real superposition, real entanglement (or deliberately none, for Experiment 1), still fully intact right up until the moment the popup renders it.

2.4. Quantum circuits for cards visualization

In this section we will see the differences in the quantum circuits for each variations of the game. Starting from the easier versions, without entanglement and going forward to most complex circuits with more quantum gates.

We will not see in details the math behind this circuits, but we will try the most common and tested versions of the circuits; that you can find also in the IBM documentation of Qiskit.

2.4.1. Experiment 1: absence of entanglement

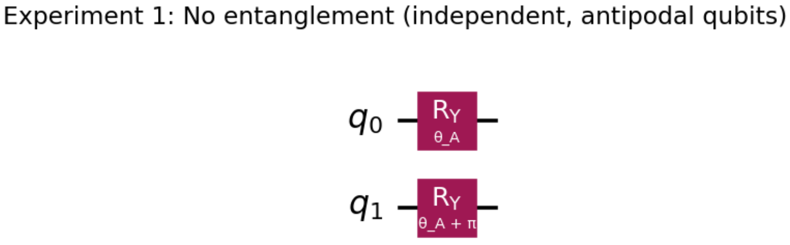

Experiment 1: No entanglement (independent, antipodal qubits)
q0
RY
θ_A
q1
RY
θ_A + π

Figure 2.3.: *two independent* **`ry`** *rotations, no entangling gate*

The circuit associated with this experiment consists of two independent single qubit rotations, each implemented by means of an R_y gate, applied respectively to the first and second qubit. No-two qubit gate is present in this circuit. The rotation angle applied to the second qubit is fixed to be the rotation angle of the first qubit augmented by π, so that *the corresponding Bloch vectors are diametrically opposed.* The rotation angle itself is obtained through a quantum random process, consistent with the method employed for deck generation, and is fixed once for each pictorial symbol at the beginning of the session. The absence of any interaction between the two qubits ensures that *the resulting two-qubit state is separable*, and therefore devoid of entanglement, independently of the specific value of the rotation angle.

This experiment establishes the reference condition against which the entangled states presented in the subsequent experiments are to be interpreted, and permits the direct observation of two qubits each possessing a fully defined individual state.

2.4.2. Experiment 2: partial and variable entanglement

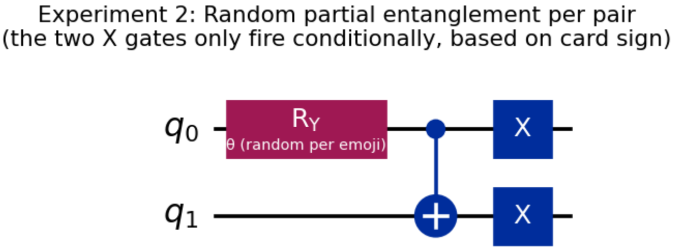

Experiment 2: Random partial entanglement per pair
(the two X gates only fire conditionally, based on card sign)
q_0
q_1
R$_Y$
θ (random per emoji)
X
X

Figure 2.4.: *partial-entanglement* `ry + cx`

The circuit associated with this experiment comprises a single qubit R_y rotation applied to the first qubit, followed by a controlled NOT gate (in the picture the cross inside the blue circle) with the first qubit as control and the second as target. Two additional single qubit NOT gates may be conditionally applied to the first and second qubit respectively, according to the sign encoded on the corresponding cards; these operations, being local, do not affect the degree of entanglement of the resulting state, but determine which of the two computational basis states is associated with a positive or negative amplitude. The rotation angle preceding the controlled NOT gate is obtained through a quantum random process and is fixed independently for each pictorial symbol. As this angle varies, the resulting two qubit state exhibits an entanglement entropy ranging continuously between zero and one ebit, with the maximal value attained when the rotation angle equals $\pi/2$.

This experiment is designed to illustrate that *entanglement admits intermediate degrees*, in contrast to the binary distinction addressed in the preceding experiment, and to demonstrate the corresponding progressive reduction in the definiteness of the individual qubit states.

2.4.3. Experiment 3: the Bell state basis

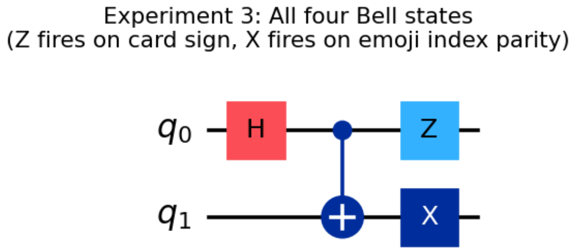

Experiment 3: All four Bell states
(Z fires on card sign, X fires on emoji index parity)
q_0
q_1
H
Z
X

Figure 2.5.: `h + cx`, *then conditional* `z/x`

The circuit associated with this experiment consists of a Hadamard gate applied to the first qubit, followed by a controlled NOT gate with the first qubit as control and the second as target, generating a *maximally entangled state.* A Z gate may be conditionally applied to the first qubit according to the sign of the corresponding card, and an X gate may be conditionally applied to the second qubit according to the parity of the index associated with the pictorial symbol. The combined application of these two conditional operations *allows the circuit to generate any one of the four Bell states.* Each of these four states possesses identical entanglement entropy, equal to one ebit, and identical marginal probability distributions for the individual qubits, yet they are mutually distinguishable through the relative phase and bit-parity structure of their amplitudes.

This experiment demonstrates that *maximal entanglement does not correspond to a single quantum state*, but to an equivalence class of states requiring additional structural information for their complete characterization.

As we have seen before, the Bell states can be obtained with different circuits, but in this experiment we are demonstrating that also with the circuit producing Φ^+ we can reach all of the other three Bell states, Φ^-, Ψ^+, and Ψ^-, without changing the entangling step itself.

To be more precise, we are using a *Bell state preparation circuit* (Hadamard followed by CNOT), circuit that produces Φ^+; followed by a *conditional Pauli encoding* step (Z and/or X, applied to one qubit only).

The idea behind this experiment is that, once two qubits have been made entangled, moving between the four Bell states does not require building the entanglement again from scratch. Starting from the same entangled pair, a small operation performed on only one of the two qubits, leaving the other completely untouched, is enough to turn the pair into a different member of the Bell family. In this experiment, that operation is decided by two pieces of information that the game already provides: the sign of the matched card, which decides whether a phase adjustment is introduced, and the identity of the matched picture, which decides whether a bit flip is introduced. Depending on which of these two conditions are met, the same starting entangled pair ends up as one of the four possible Bell states.

What makes this worth highlighting from an educational standpoint is twofold. First, it shows that maximal entanglement does not describe a single, unique state of two qubits, but rather an entire family of four states, all equally and maximally entangled, and distinguished from each other only by these small, local adjustments. Second, and more strikingly, it shows that acting on just one of the two qubits is sufficient to change which member of that family the pair becomes, even though the two qubits remain entangled throughout, and even though the second qubit is never directly touched. This has no real counterpart in classical physics, where performing an operation on one object cannot change how that object is jointly described together with another, distant object. It is precisely this property, of steering a shared entangled pair by acting on only one half of it, that underlies well-known quantum communication protocols such as superdense coding, in which a sender is able to communicate one of four possible messages to a receiver by transmitting a single qubit, relying on an entangled pair the two parties already share in advance.

2.4.4. Experiment 4: entanglement grows with pairs found

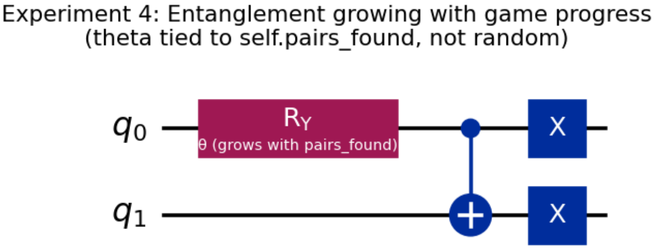

Experiment 4: Entanglement growing with game progress
(theta tied to self.pairs_found, not random)
q_0
q_1
R$_Y$
θ (grows with pairs_found)
X
X

Figure 2.6.: *same shape as Experiment 2, but* `theta` *tied to game progress*

The circuit associated with this experiment is structurally identical to that of the second experiment, comprising a single qubit R$_y$ rotation applied to the first qubit, a controlled NOT gate coupling the first and second qubit, and two conditionally applied NOT gates encoding the sign of the corresponding cards. The distinguishing feature of this experiment lies not in the circuit itself, but in the determination of the rotation angle, which is not obtained through a random process but is instead computed as a function of the number of matching pairs previously identified during the session, increasing monotonically from zero to $\pi/2$. Consequently, the entanglement entropy of the generated state increases monotonically over the course of the session, from 0 to 1 ebit.

This experiment is intended to convey the notion of entanglement as a *dynamically evolving quantity*, associated with the progression of the game, rather than as a static or randomly determined property.

2.4.5. Experiment 5: entanglement not manifest in measurement probabilities

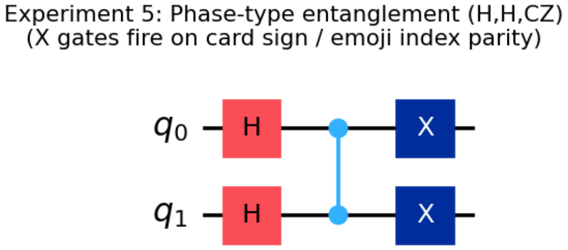

Experiment 5: Phase-type entanglement (H,H,CZ)
(X gates fire on card sign / emoji index parity)
q_0
q_1
H
H
X
X

Figure 2.7.: *`h`, `h`, `cz`, then conditional `x`*

The circuit associated with this experiment consists of a Hadamard gate applied to each of the two qubits, followed by a controlled Z gate. Two NOT gates may be conditionally applied, respectively to the first and second qubit, according to the sign of the corresponding card and the parity of the index of the associated pictorial symbol. The resulting state is *maximally entangled*, with an entanglement entropy equal to one ebit; however, the probability distribution obtained upon measurement in the computational basis is uniform across all four possible outcomes, irrespective of the specific state generated. The entanglement is manifest exclusively through the correlation of measurement outcomes obtained in a different measurement basis, specifically along the Y axis.

This experiment demonstrates that the detection of entanglement is contingent upon the choice of measurement basis, and that a uniform outcome distribution in one basis does not, in general, permit the inference of separability of the underlying state.

3. Qubit Visualization

3.1. Overview

The traditional approach to teaching quantum computing typically requires the learner, to first acquire a substantial body of formal mathematics, including: linear algebra, complex numbers, and probability theory, before any intuition about the behavior of qubits can be developed. This sequencing places a considerable barrier between the learner and the subject matter, and often results in the physical and conceptual significance of concepts such as *superposition* and *entanglement* being obscured by the notation required to describe them. The pedagogical premise underlying the present work is that this barrier can be substantially lowered by inverting the customary order of exposure: rather than deriving intuition from formalism, the learner is first placed in direct, repeated, and low stakes contact with the phenomena themselves, with the formal description introduced only afterwards, *as an explanation of what has already been observed.*

Play is intrinsically motivating, and therefore sustains attention over the repeated trials that are necessary for an unfamiliar and counter-intuitive concept to become familiar. Play is also forgiving of error: within the context of a game, an unexpected or seemingly paradoxical outcome is not experienced as a failure to be avoided, but as an invitation to further exploration, in a manner that is considerably more difficult to reproduce within a formal instructional setting.

As we have seen in the past chapter to learn more about quantum entanglement I have created different circuits, like *levels in a video game*; to discover better the behaviors behind different configurations of circuits. In the following pages I'll show you the visualizations for each experiments explaining the different properties observed in the simulation and also running the code on the *real quantum computer.*

To visualize the data I have used different Python modules, explained in details in the next chapter. Not always will be possible to see the full complexity of all the aspects of qubits, because the evolution of quantum computers is still ongoing trying to remove the bias generated from the noise of QPU; the evolution of quantum computers is still ongoing, trying to reduce the errors generated by the noise of the QPU. Some companies are also developing small quantum processors based on defects in synthetic diamond that, according to the manufacturer, *can work at room temperature* [25]; these devices are still at an early stage and have only a few qubits.

With this experiments I'm trying to emphasize only new horizons shaped by the entanglement state between two qubits; but to be more exhaustive and intellectually honest I have to mention also that is possible to establish the entanglement state with more then two qubits. It's also necessary to specify to the reader that since the two qubits are connected some operations characteristics of the qubits will not be visible. To show this compromise about learning and playing, I have deliberately chosen to don't create any entanglement in the first experiment, as way to make the reader consciousness about how it is appearing a qubit in a Bloch sphere.

3.2. Reading graphs: complex numbers, vectors, and the Bloch sphere

To better understand the graph and chart produced during the experiments I have added here, a simple reminder of the concepts needed to generate the charts and understand it.

3.2.1. Complex numbers as vectors

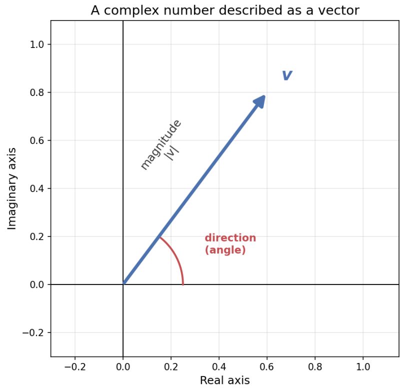


Figure 3.1.: *graph of a complex number as vector*

A complex number can be described, for the purposes of this thesis, in exactly the same terms as an ordinary vector in physics: a magnitude, and a direction. The figure that you can see up there (Figure 3.1), illustrates this correspondence. A complex number $a + bi$ is drawn as an arrow, denoted v, originating from the origin of a plane; its length, denoted $|v|$ and referred to as its magnitude, and its direction, given by the angle the arrow makes with the horizontal axis, together fully specify it. This is the same descriptive language used for vectors elsewhere in physics, and it is the only language required to interpret the amplitudes appearing throughout this thesis.

3.2.2. Describing a vector by its numbers

Alongside the picture of an arrow, a vector can be written just as simply as a short list of plain numbers, and this is the form in which vectors actually appear in the figures generated by the game. For a vector drawn in a plane, two numbers are enough: one telling how far along the horizontal axis the arrow reaches, and one telling how far along the vertical axis it reaches. This is exactly what is meant, for example, by a line such as |00⟩: −0.49+0.00i, printed in several of the figures: it states that the arrow representing that particular amplitude reaches −0.49 along the horizontal axis, and 0.00 along the vertical axis, that is, it points directly along the horizontal axis, in the negative direction, at a distance of 0.49 from the origin.

The vector used on the Bloch sphere works in the same way, except that three numbers are needed instead of two, one for each of the three directions in which the sphere extends. These three numbers are what the game computes internally in order to draw the arrow shown in each Bloch sphere picture, and they are also what allows the length of that arrow to be measured: the longer the arrow, the larger these numbers are, and when the arrow shrinks to the center of the sphere, all three numbers approach zero. The value printed as */v/* in the figures is precisely this length, obtained from those three numbers.

3.2.3. Vector length and probability

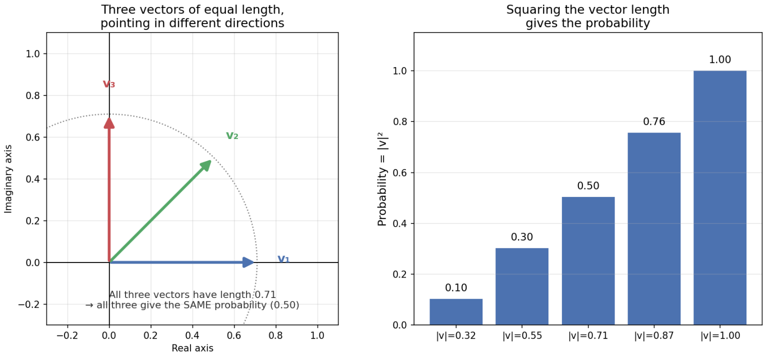


Figure 3.2.: *vectors and probability*

Each of the two basic states of a qubit, |0⟩ and |1⟩, appears in a superposition together with a vector of this kind, referred to as an amplitude. The rule connecting an amplitude to anything observable concerns only its magnitude: the probability of obtaining a given outcome upon measurement is equal to the square of the magnitude of the corresponding vector. This is illustrated in the right-hand panel of figure (Figure 3.2), where vectors of increasing length are shown alongside the probability obtained by squaring that length; a vector of magnitude 0.71, repeatedly encountered in the figures generated by the game, corresponds to a probability of approximately 0.50.

An important consequence of this rule, illustrated in the left side panel of the figure, is that the probability depends only on the length of the vector, and not on its direction. The three example vectors (red, green and blue arrows) shown that are of equal length, but are pointing in three different directions, and all three consequently correspond to the same probability of 0.50. The direction of the vector, referred to as its phase, is therefore not reflected in the probability value alone; it constitutes separate information, which will be shown in later sections to be responsible for distinguishing between certain quantum states that would otherwise appear identical.

3.2.4. The Bloch sphere

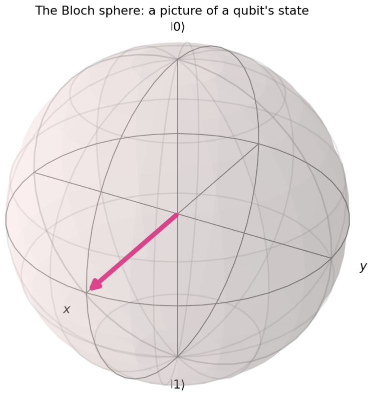


Figure 3.3.: *the Bloch sphere*

The Bloch sphere, shown in figure (Figure 3.3), extends this vector description to the state of a qubit considered as a whole. The state is represented as a single vector drawn from the center of a sphere to a point on its surface: a vector pointing to the top of the sphere corresponds to the state $|0\rangle$, a vector pointing to the bottom corresponds to the state $|1\rangle$, and, as shown in the figure, a vector pointing on the equator (the x in the figure), midway between the two, corresponds to an equal blend of both. Every direction on the sphere corresponds to a possible state of the qubit, with the two poles representing the two definite classical outcomes and every other point representing some intermediate superposition of them.

The length of this vector is equally significant, and constitutes the detail of greatest relevance to the figures presented in this thesis. *A vector reaching the surface of the sphere indicates that the qubit possesses a fully defined state of its own.* When two qubits become entangled, however, neither of them retains such a state considered individually; the state can only be properly described by referring to the pair as a whole. This circumstance is reflected, in the figures generated by the game, by a visible shortening of the corresponding vector, down to the exact center of the sphere in the case of maximal entanglement, at which point the vector vanishes entirely. The presence of a full-length vector therefore indicates that a qubit is not entangled, while a shortened or vanished vector indicates that it is, with the degree of shortening reflecting the extent of the entanglement involved.

3.3. Play and learn

The game is easy! The player have simply to find two cards with the same emoji! Once done, if there is a match; a graph with all information about the qubit and entanglement state will be visualized on the screen.

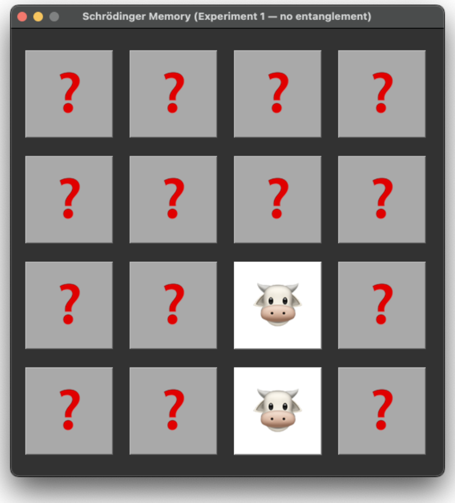
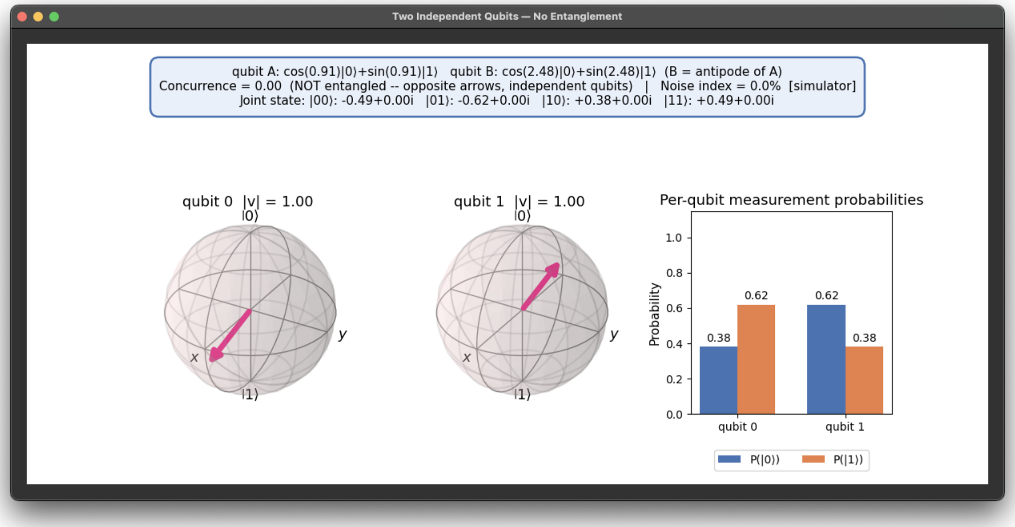


Figure 3.4.: *match of two cards*

For instance in this case we have found two cows, and and a pop up showing the related information about what is happening behind the screen will be visualized. In the next chapters we will examine the information in detail for each experiment.

3.3.1. Experiment 1: two independent qubits

Finding a couple of cards with the same emoji in the first experiments show us a graphical view of two qubits without entanglement as follow:



Two Independent Qubits — No Entanglement

```
qubit A: cos(0.91)|0⟩+sin(0.91)|1⟩   qubit B: cos(2.48)|0⟩+sin(2.48)|1⟩  (B = antipode of A)
Concurrence = 0.00  (NOT entangled -- opposite arrows, independent qubits)   |   Noise index = 0.0%  [simulator]
Joint state: |00⟩: -0.49+0.00i   |01⟩: -0.62+0.00i   |10⟩: +0.38+0.00i   |11⟩: +0.49+0.00i
```

Figure 3.5.: *two independent qubits*

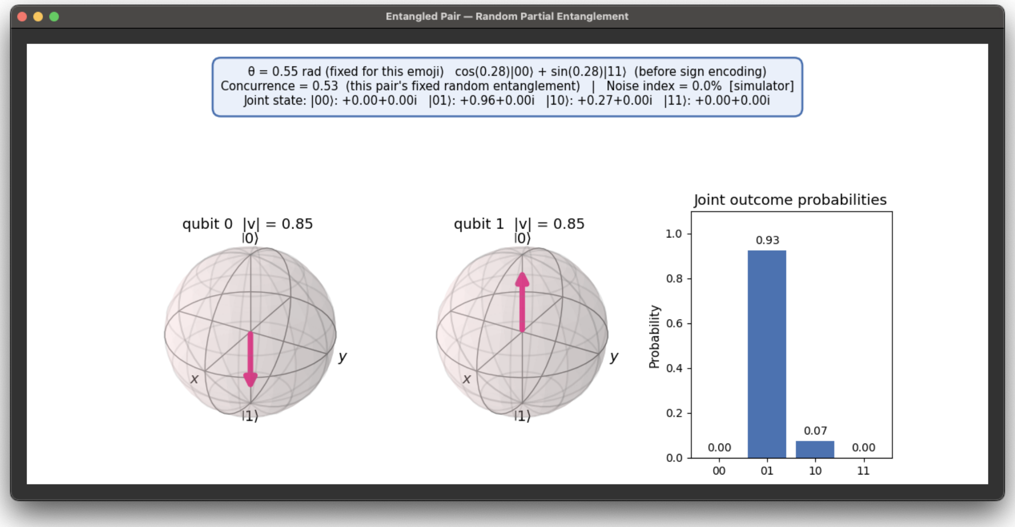
The uppermost region (Figure 3.5) presents a summary of the quantum state associated with the two qubits involved. The individual states of qubit A and qubit B are given explicitly, in the form $cos(\vartheta)/0\rangle + sin(\vartheta)/1\rangle$, with the rotation angle of qubit B differing from that of qubit A by π, in accordance with the antipodal relationship established between the two qubits in this experiment. Immediately below, the concurrence of the pair is reported as exactly zero, together with an explicit indication that the *two qubits are not entangled*, and a noise index of zero per cent, confirming that the state was generated by an ideal, noiseless simulation rather than by execution on physical hardware. The final line of this region reports the four complex amplitudes of the joint two qubit state in the computational basis.

The central region displays the Bloch sphere representation of each of the two qubits individually. Both vectors are reported with unit magnitude, denoted $/v/ = 1.00$, indicating that each qubit occupies a pure state; the two vectors are observed to point in diametrically opposite directions, a direct visual consequence of the angular relationship imposed between them. *This is to be contrasted with the qubit visualizations produced in the entangled experiments, in which the corresponding vectors are of reduced magnitude, or vanish entirely in the case of maximal entanglement.*

The rightmost region presents the measurement probabilities of each qubit individually in the computational basis, obtained directly from the corresponding Bloch vector. The values displayed, approximately 0.38 and 0.62 for qubit 0 and their exact reverse for qubit 1, are found to be numerically consistent with the rotation angles reported in the upper region, and are symmetric between the two qubits as a further consequence of their antipodal relationship. The absence of any bar corresponding to a joint or correlated outcome is notable, and distinguishes this panel from the joint probability representations employed in the entangled experiments, reflecting the fact that, in the absence of entanglement, each qubit possesses an independently meaningful probability distribution of its own.

Taken together, the figure constitutes direct visual evidence, obtained from the running application rather than from a purely theoretical construction, of a separable two-qubit state: two qubits in well-defined, individually observable configurations, exhibiting no measurable correlation, and no entanglement as quantified by the reported concurrence.

3.3.2. Experiment 2: partially entangled vectors



```
θ = 0.55 rad (fixed for this emoji)   cos(0.28)|00⟩ + sin(0.28)|11⟩  (before sign encoding)
Concurrence = 0.53  (this pair's fixed random entanglement)   |   Noise index = 0.0%  [simulator]
Joint state: |00⟩: +0.00+0.00i   |01⟩: +0.96+0.00i   |10⟩: +0.27+0.00i   |11⟩: +0.00+0.00i
```

Figure 3.6.: *two partially entangled qubits*

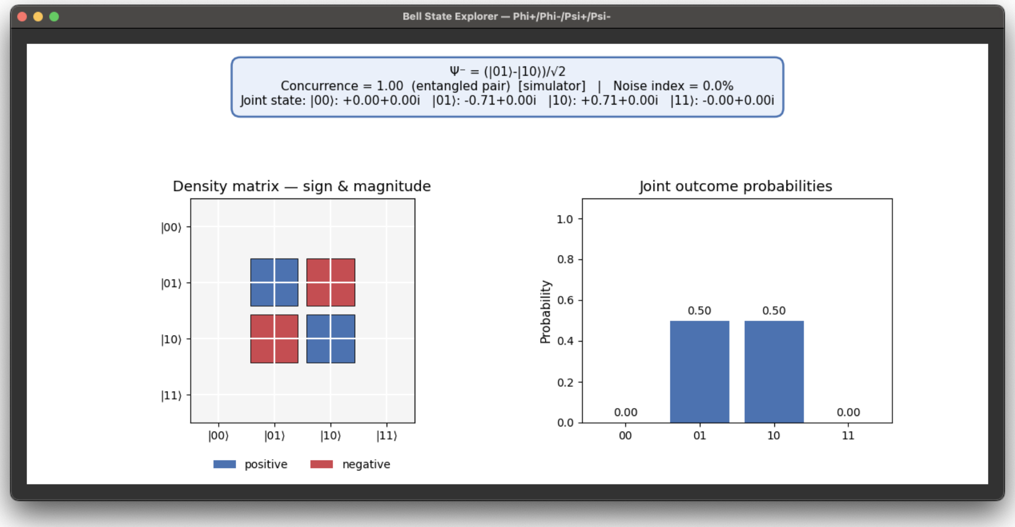
The figure (Figure 3.6) reproduces a screenshot of the graphical interface generated by Experiment 2 following the identification of a matching pair of cards during actual execution of the application.

The uppermost region reports the parameters and outcome of the partially entangled circuit. The rotation angle ϑ, here equal to 0.55 radians and fixed once for the emoji associated with this pair, is given together with the corresponding pre-encoding state, *cos(0.28)|00⟩ + sin(0.28)|11⟩*. The concurrence of the resulting pair is reported as 0.53, explicitly labeled as this particular pair's fixed degree of entanglement, together with a noise index of zero per cent, confirming that the state was obtained from an ideal, noiseless simulation. The final line reports the four complex amplitudes of the joint two qubit state; it may be observed that the amplitude is concentrated on the outcomes $|01\rangle$ and $|10\rangle$, rather than $|00\rangle$ and $|11\rangle$ as the pre-encoding formula would suggest, a consequence of the local bit-flip operation applied during the encoding of the card's sign, discussed in the preceding sections.

The central region displays the Bloch sphere representation of the two qubits individually. In contrast to the case of separable qubits, the two vectors are here observed to be of reduced length, each reported as $|v| = 0.85$, rather than reaching the surface of the sphere. This value is consistent with the reported concurrence, the two quantities being related in such a way that an increase in one corresponds to a decrease in the other; a partially entangled pair is thus seen to correspond to a partially shortened vector, of *intermediate length between the fully defined vectors of a separable qubit and the vanishing vectors of a maximally entangled pair.*

The rightmost region presents the joint outcome probabilities, that is, the probabilities associated with the four possible results of measuring both qubits together, rather than the probabilities of each qubit considered individually. The distribution is found to be concentrated principally on the outcome $|01\rangle$, with probability 0.93, and to a lesser extent on the outcome $|10\rangle$, with probability 0.07, while the outcomes $|00\rangle$ and $|11\rangle$ are found to occur with negligible probability. These values are consistent with the squared magnitudes of the corresponding amplitudes reported in the upper region. *The presence of two, rather than four, distinguishable outcomes with appreciable probability constitutes a further indication of the correlation existing between the two qubits, in contrast with the case of a separable pair, for which all four outcomes would in general occur with comparable probability.*

3.3.3. Experiment 3: Bell states and density matrix



```
Ψ⁻ = (|01⟩-|10⟩)/√2
Concurrence = 1.00  (entangled pair)  [simulator]   |   Noise index = 0.0%
Joint state: |00⟩: +0.00+0.00i   |01⟩: -0.71+0.00i   |10⟩: +0.71+0.00i   |11⟩: -0.00+0.00i
```

Bell State Explorer — Phi+/Phi-/Psi+/Psi-
Density matrix — sign & magnitude
|00⟩
|01⟩
|10⟩
|11⟩
|00⟩
|01⟩
|10⟩
|11⟩
positive
negative
Joint outcome probabilities
Probability
1.0
0.8
0.6
0.4
0.2
0.0
0.00
0.50
0.50
0.00
00
01
10
11

Figure 3.7.: *Bell state generator*

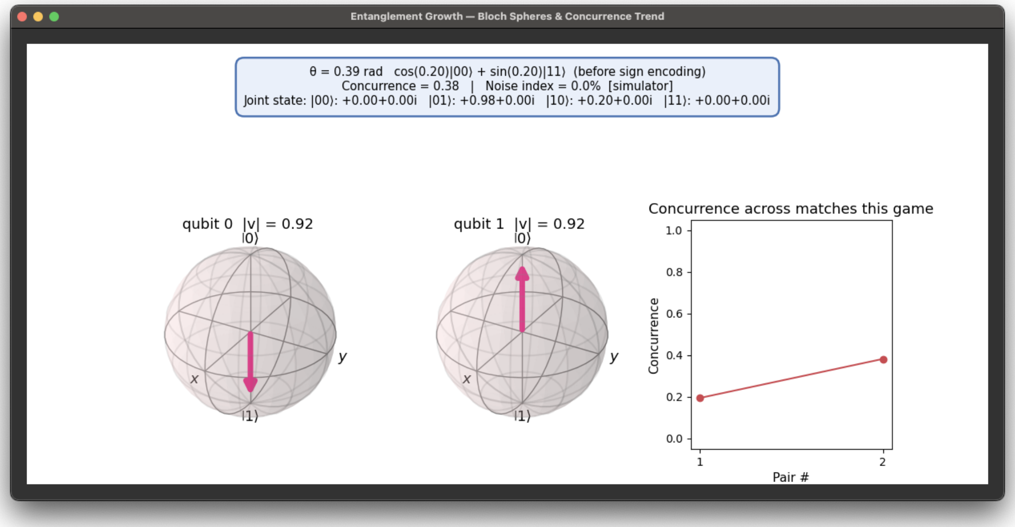
The figure (Figure 3.7 on the preceding page) reproduces a screenshot of the graphical interface generated by Experiment 3 following the identification of a matching pair of cards during actual execution of the application. The window departs from the layout employed in the preceding experiments in that the two Bloch sphere representations are here replaced by a color coded representation of the density matrix associated with the generated state, positioned alongside the joint outcome probability chart.

The uppermost region identifies the specific Bell state produced on this occasion as $\Psi^- = (|01\rangle - |10\rangle)/\sqrt{2}$, together with a concurrence of 1.00, explicitly labeled as corresponding to an entangled pair, and a noise index of zero per cent, confirming that the state was obtained from an ideal, noiseless simulation. The four complex amplitudes of the joint state are reported in the final line, and are found to be consistent with the stated formula, with non-zero amplitude, of equal magnitude and opposite sign, associated with the outcomes $|01\rangle$ and $|10\rangle$.

The density matrix representation, shown in the lower left, employs a grid in which each cell corresponds to one entry of the matrix, indexed by a pair of basis states, $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$. *The size of each colored square is proportional to the* ***magnitude*** *of the corresponding matrix entry, while its color indicates the sign of that entry, blue denoting a positive value and red a negative value.* In the present case, non-zero entries are observed exclusively within the block corresponding to the $|01\rangle$ and $|10\rangle$ states, consistent with the Bell state identified above; within this block, the entries lying on the diagonal are positive, while those lying off the diagonal are negative, this specific pattern of signs being the distinguishing feature that separates the state Ψ^- from the state Ψ^+, the two states sharing an otherwise identical distribution of magnitudes.

The joint outcome probability chart, shown in the lower right, reports probabilities of 0.50 for each of the outcomes $|01\rangle$ and $|10\rangle$, and a probability of zero for the outcomes $|00\rangle$ and $|11\rangle$, consistent with the amplitudes reported above. *It may be observed that this distribution is indistinguishable from the one that would be obtained for the state Ψ^+, since the two states differ only in the relative sign of their amplitudes, a feature to which the measurement probabilities, depending as they do only on the squared magnitude of each amplitude, are not sensitive.* The distinction between the two states is accordingly not visible in this panel, and is recoverable only from the density matrix representation positioned alongside it, this contrast constituting the principal illustrative purpose of the present figure.

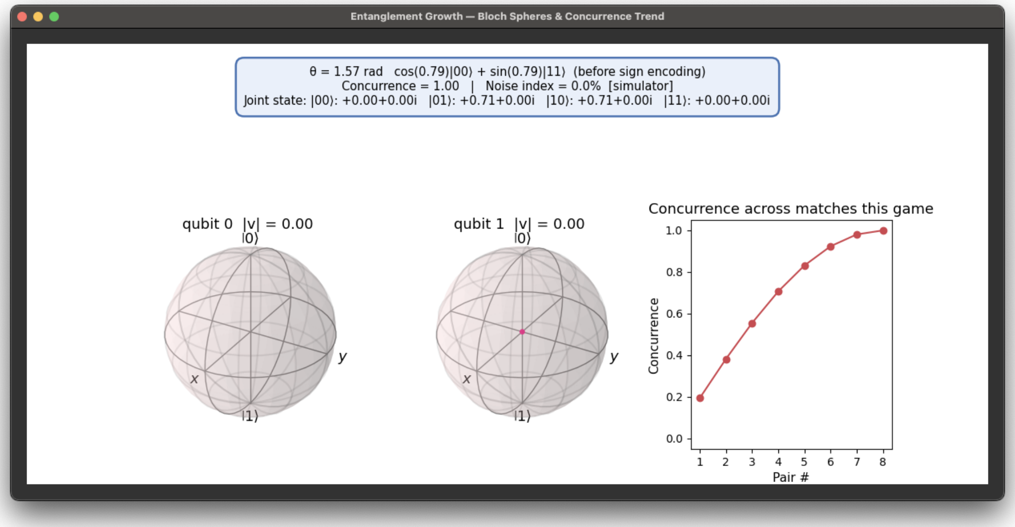
3.3.4. Experiment 4: progressive increase of entanglement



```
θ = 0.39 rad   cos(0.20)|00⟩ + sin(0.20)|11⟩  (before sign encoding)
Concurrence = 0.38   |   Noise index = 0.0%  [simulator]
Joint state: |00⟩: +0.00+0.00i   |01⟩: +0.98+0.00i   |10⟩: +0.20+0.00i   |11⟩: +0.00+0.00i
```

Entanglement Growth — Bloch Spheres & Concurrence Trend
qubit 0 |v| = 0.92
|0⟩
x
y
|1⟩
qubit 1 |v| = 0.92
|0⟩
x
y
|1⟩
Concurrence across matches this game
Concurrence
1.0
0.8
0.6
0.4
0.2
0.0
1
2
Pair #

Figure 3.8.: *progressive entanglement after the second match*



```
θ = 1.57 rad   cos(0.79)|00⟩ + sin(0.79)|11⟩  (before sign encoding)
Concurrence = 1.00   |   Noise index = 0.0%  [simulator]
Joint state: |00⟩: +0.00+0.00i   |01⟩: +0.71+0.00i   |10⟩: +0.71+0.00i   |11⟩: +0.00+0.00i
```

Entanglement Growth — Bloch Spheres & Concurrence Trend
qubit 0 |v| = 0.00
|0⟩
x
y
|1⟩
qubit 1 |v| = 0.00
|0⟩
x
y
|1⟩
Concurrence across matches this game
Concurrence
1.0
0.8
0.6
0.4
0.2
0.0
1
2
3
4
5
6
7
8
Pair #

Figure 3.9.: *progressive entanglement after the eighth match*

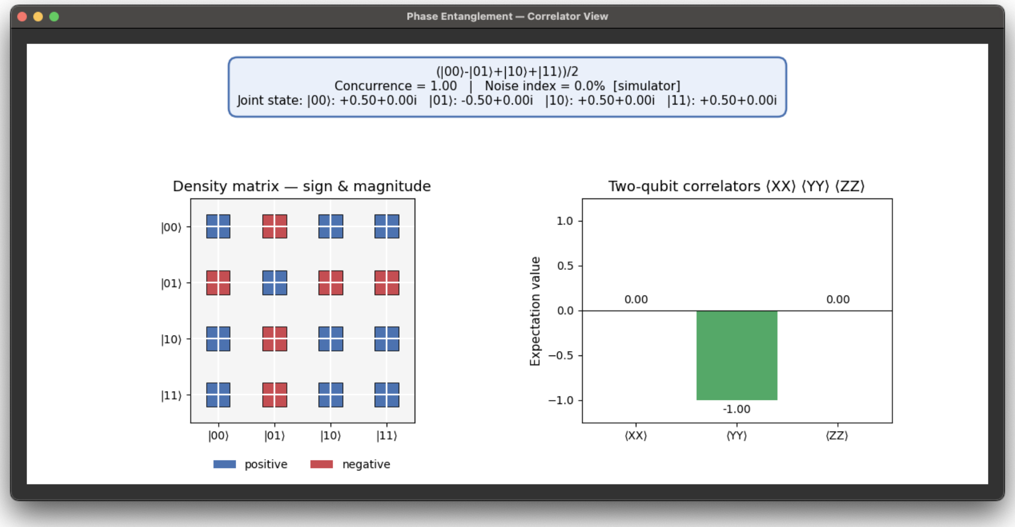
The two figures (Figure 3.8 and Figure 3.9) reproduce screenshots of the graphical interface generated by Experiment 4 at two different points within the same game session, namely following the second and the eighth, and final, successful match, respectively. Both are organized according to the same layout employed throughout this thesis, with the addition, in the rightmost panel, of a plot showing the concurrence obtained at each successive match within the session, in place of the outcome probability charts employed in the preceding experiments.

In the first figure (Figure 3.8 on the facing page), corresponding to the second match of the session, the rotation angle ϑ is reported as 0.39 radians, a value determined by the number of pairs previously identified rather than by a random process, as is the case in this experiment. The corresponding concurrence is reported as 0.38, and the two Bloch vectors are each of length 0.92, a value consistent with a comparatively low degree of entanglement. The trend panel on the right displays two points, corresponding to the first and second matches of the session, showing an increase in concurrence from approximately 0.20 to 0.38.

In the second figure (Figure 3.9 on the preceding page), corresponding to the eighth and final match of the session, the rotation angle ϑ is reported as 1.57 radians, corresponding to $\pi/2$, at which the entangling operation produces a *maximally entangled state*. The concurrence is accordingly reported as 1.00, and the length of each Bloch vector is reported as 0.00, both vectors having contracted to the center of the sphere; a residual marker is visible at the center of the sphere associated with the second qubit, corresponding to the same vanishing vector, of exactly zero length. The trend panel on the right now displays eight points, one for each match of the session, showing a monotonic increase in concurrence from approximately 0.20 at the first match to exactly 1.00 at the eighth.

Considered together, the two figures document the defining characteristic of this experiment, namely the progressive increase of entanglement over the course of a single game session, rather than its determination by an independent random process, as in the case of the preceding experiment. *The correspondence between the shortening of the two Bloch vectors and the rise of the plotted concurrence curve, observable across the two figures, provides direct visual confirmation of the relationship between these two quantities discussed in the preceding sections.*

3.3.5. Experiment 5: maximally entangled state exhibiting



(|00⟩-|01⟩+|10⟩+|11⟩)/2
Concurrence = 1.00 | Noise index = 0.0% [simulator]
Joint state: |00⟩: +0.50+0.00i |01⟩: -0.50+0.00i |10⟩: +0.50+0.00i |11⟩: +0.50+0.00i

Figure 3.10.: *maximally entangled state qubits*

The figure (Figure 3.10) reproduces a screenshot of the graphical interface generated by Experiment 5 following the identification of a matching pair of cards during actual execution of the application. The window, as in the preceding figure, replaces the two Bloch sphere representations with a color coded representation of the density matrix; in this case, however, the joint outcome probability chart employed elsewhere is in turn replaced by a chart of two qubit correlation.

The uppermost region identifies the generated state through the equation explained with the relation between this qubits, $(|00\rangle - |01\rangle + |10\rangle + |11\rangle)/2$, together with a concurrence of 1.00 and a noise index of zero per cent, confirming a maximally entangled state obtained from an ideal, noiseless simulation. The four complex amplitudes reported in the final line are found to be of equal magnitude, 0.50, differing only in the sign associated with the outcome $|01\rangle$, consistent with the stated formula.

The density matrix representation, shown in the lower left, exhibits a pattern markedly different from that observed in the corresponding figure for the preceding experiment: rather than being confined to a single block of the matrix, non-zero entries, of equal magnitude, are present throughout the entire grid, each of the sixteen cells being populated with either a positive or a negative value. *This density matrix therefore corresponds to a state in which every one of the four basis outcomes contributes to the superposition, in contrast with the Bell states considered previously, in which only two basis outcomes were involved.*

The chart shown in the lower right reports the expectation values of the three two qubit correlation, ⟨XX⟩, ⟨YY⟩, and ⟨ZZ⟩, corresponding to measurements performed in three different combinations of measurement bases. The correlation ⟨XX⟩ and ⟨ZZ⟩ are both found to be exactly zero, whereas the correlation ⟨YY⟩ is found to attain its most negative possible value, -1.00. This outcome is to be contrasted with the joint outcome probability chart that would be obtained for this same state upon measurement in the standard computational basis, which, on account of the uniform magnitude of the four amplitudes reported above, would assign equal probability, 0.25, to each of the four possible outcomes, and would accordingly exhibit no discernible structure. The correlation chart shown here demonstrates that the entanglement present in this state, despite being maximal, is not manifest in the computational basis, and becomes observable only when measurements are performed in an alternative basis, specifically along the Y axis of each qubit, this circumstance constituting the principal illustrative purpose of the present experiment.

4. Experiments on a real quantum computer

4.1. Overview

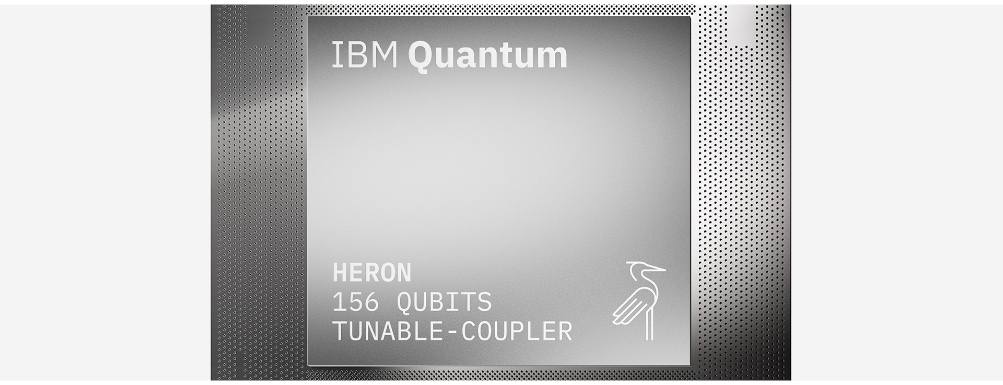
The experiments that I have made has been realized with the IBM quantum platform. The creation of a new account on the platform give to every user 10 mins of free quantum computing, to complete all the tests I have used circa 15 mins.

An important aspect to take in consideration that could be useful to all the people that want to approach this kind of experiments is that the time of consuming for the computing start at the run of your script and it include also the wait time for the free resources assigned to run the Qiskit script.

This means that the free 10 mins of utilization given with the free account is not the effective time that you will be used by the quantum processor unit.

4.2. The quantum processor unit or QPU

The server where I ran my Qiskit scripts was a server equipped with the processor *Heron r2* with with 156 programmable qubits and I have made all my test with 1024 number of shots. The free account was enough to run my experiments but it could be a big limitation for the students or technicians that want to push a little bit more the research in this field.



> *The combined improvements across IBM Heron and Qiskit can execute certain mirrored kicked Ising quantum circuits of up to 5,000 gates, which is nearly twice the number of gates accurately run in IBM's 2023 demonstration of quantum utility. This work further extends the performance of IBM's quantum computers beyond the capabilities of brute-force classical simulation methods. The 2023 utility experiment, published in Nature, demonstrated the speed results in terms of time to process, per data point, which totaled 112 hours. The same experiment, using the same data points, was run on the latest IBM Heron processor and can be completed in 2.2 hours, which is 50 times faster.*
>
> *IBM has further evolved Qiskit into the world's most performant quantum software to allow developers to more easily build complex quantum circuits with stability, accuracy, and speed. This is evidenced by results gathered and published on arXiv.org using Benchpress, an open-source benchmarking tool which IBM used to measure Qiskit across 1,000 tests, largely from third parties, and found it to be the highest-performing, most reliable quantum software development kit against other selected platforms.*
>
> [26, IBM Launches Its Most Advanced Quantum Computers]

4.3. Real runnings = real results

In this chapter I will show the results obtained with the real running of my scripts in the IBM Quantum Platform. An important consideration that have to be made is that in some case the results will be really different from the simulations because sometimes the real hardware can't give us some data like the full state of a vector.

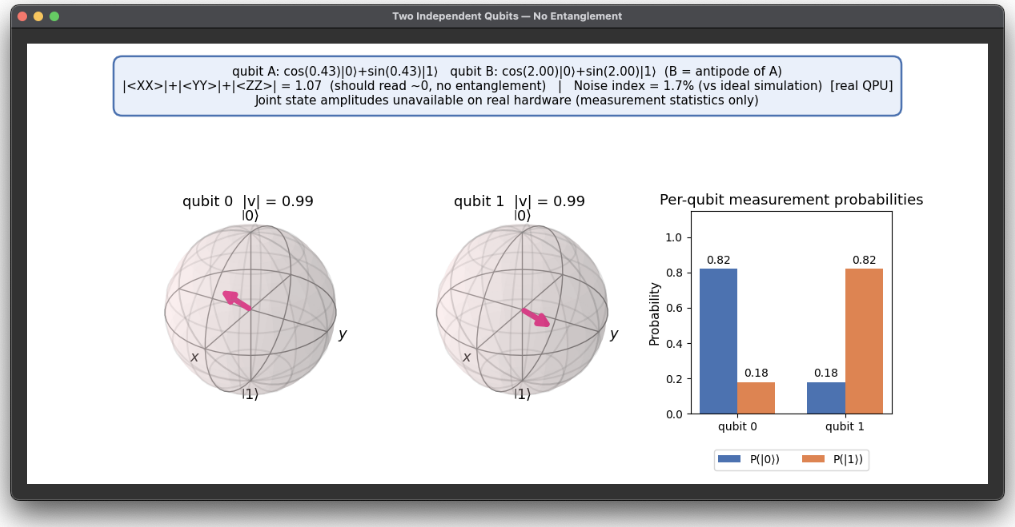
A real QPU can't give the full state because every measurement destroys it, it also give only one bit per qubit each time.

The real hardware make a measurement for each run of the circuit. The measurement forces each qubit to a definite 0 or 1 and erases the superposition. You get a string like "01" and nothing else. The amplitudes (the complex numbers in front of $|00\rangle$, $|01\rangle$ and so on) are never read out directly.

The simulator is a program that stores the whole state vector in memory, so it can simply print it. A physical device has no such memory. The state exists only in the physics of the qubits, and you cannot copy it (the *no-cloning theorem [27]*) or look at it without disturbing it.

Another parameter that we will see during in the results it the **witness;** in quantum information theory, an entanglement witness is a functional which distinguishes a specific entangled state from separable ones.

4.3.1. Experiment 1: two independent qubits



```
qubit A: cos(0.43)|0⟩+sin(0.43)|1⟩   qubit B: cos(2.00)|0⟩+sin(2.00)|1⟩  (B = antipode of A)
|<XX>|+|<YY>|+|<ZZ>| = 1.07  (should read ~0, no entanglement)   |   Noise index = 1.7% (vs ideal simulation)  [real QPU]
Joint state amplitudes unavailable on real hardware (measurement statistics only)
```

Figure 4.1.: *two independent qubits with QPU*

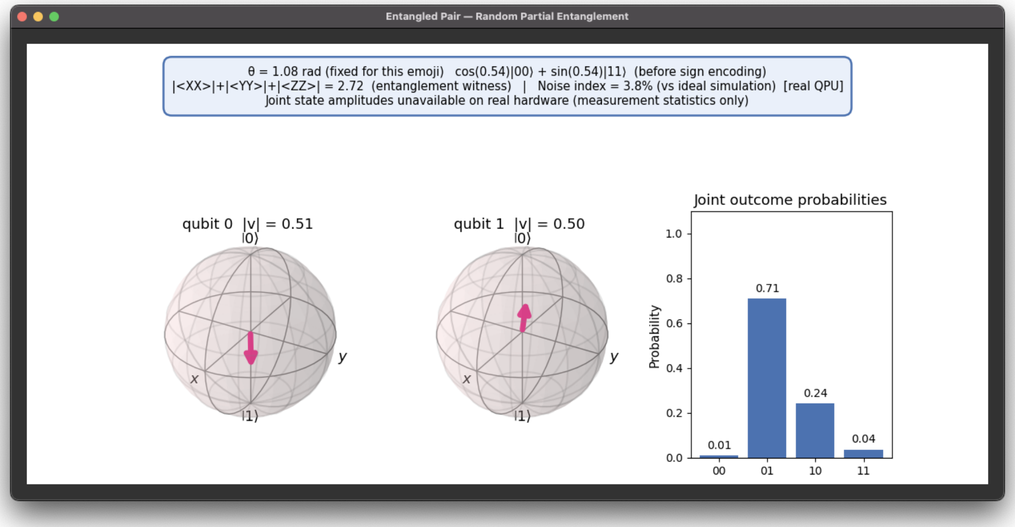
The figure (Figure 4.1) shows the result window of Experiment 1 when the circuit is executed on a real IBM quantum processor instead of the ideal simulator. The title box reports the two prepared states, qubit A with angle 0.43 rad and qubit B with angle 2.00 rad, which are antipodal by construction. The second line reports the entanglement witness, defined as the sum of the absolute values of the three two-qubit correlators, together with the noise index. The third line states that the joint state amplitudes are not available on real hardware, because a physical device returns only measurement statistics and not the underlying state vector. *This is the main difference from the simulator version of the same window, where the amplitudes of the four basis states are printed explicitly.*

The measured witness is 1.07. For two independent qubits prepared as antipodal states, the ideal value computed from the Bloch vectors is 1.00 ($\langle \mathrm{XX} \rangle \approx -0.57$, $\langle \mathrm{YY} \rangle = 0$, $\langle \mathrm{ZZ} \rangle \approx -0.43$), and 1 is also the upper bound for any separable state. The measured value exceeds this bound by only 0.07, a deviation compatible with statistical fluctuation over a finite number of shots and with readout errors. *It cannot be interpreted as entanglement, since the circuit contains no entangling gate.* The noise index is 1.7%, which corresponds to an average distance of about 0.03 between the measured and the ideal Bloch vectors, on a scale where the maximum possible distance is 2. The deviation from the ideal simulation is therefore small.

The two Bloch spheres show vectors of length 0.99 for both qubits. The simulator gives exactly 1.00, because the states are pure. *The slight reduction is the direct signature of* *hardware noise, since decoherence and gate errors turn a pure state into a slightly mixed one, which moves the vector inside the sphere.* The two arrows remain approximately opposite to each other, as required by the antipodal construction. The probability chart shows P(|0⟩) = 0.82 and P(|1⟩) = 0.18 for qubit 0, and the mirrored values for qubit 1. The ideal values for these angles are $\cos^2(0.43) \approx 0.82$ and $\sin^2(0.43) \approx 0.18$, so the agreement is within the displayed precision. As in the simulator, the two probability pairs are exact mirror images of each other, which confirms that the qubits behave independently.

In conclusion, the real processor reproduces the behavior of the simulator for Experiment 1. The vectors lose only about 1% of their length, the probabilities are essentially unchanged, and the witness stays close to the separable bound. This experiment can therefore serve as a baseline for estimating the intrinsic noise of the device. It is the reference against which the larger deviations of the entangled experiments can be judged.

4.3.2. Experiment 2: partially entangled vectors



θ = 1.08 rad (fixed for this emoji) cos(0.54)|00⟩ + sin(0.54)|11⟩ (before sign encoding)
|<XX>|+|<YY>|+|<ZZ>| = 2.72 (entanglement witness) | Noise index = 3.8% (vs ideal simulation) [real QPU]
Joint state amplitudes unavailable on real hardware (measurement statistics only)

Figure 4.2.: *two partially entangled qubits with QPU*

The figure (Figure 4.2) shows the result window of Experiment 2 when the circuit is executed on a real IBM quantum processor. The title box reports the rotation angle assigned to the matched emoji, $\vartheta = 1.08$ rad, and the corresponding state before the sign encoding, $\cos(0.54)|00\rangle + \sin(0.54)|11\rangle$. The second line reports the entanglement witness, defined as the sum of the absolute values of the three two-qubit correlators, together with the noise index. The third line states that the joint amplitudes are not available on real hardware, since only measurement statistics can be obtained from a physical device.

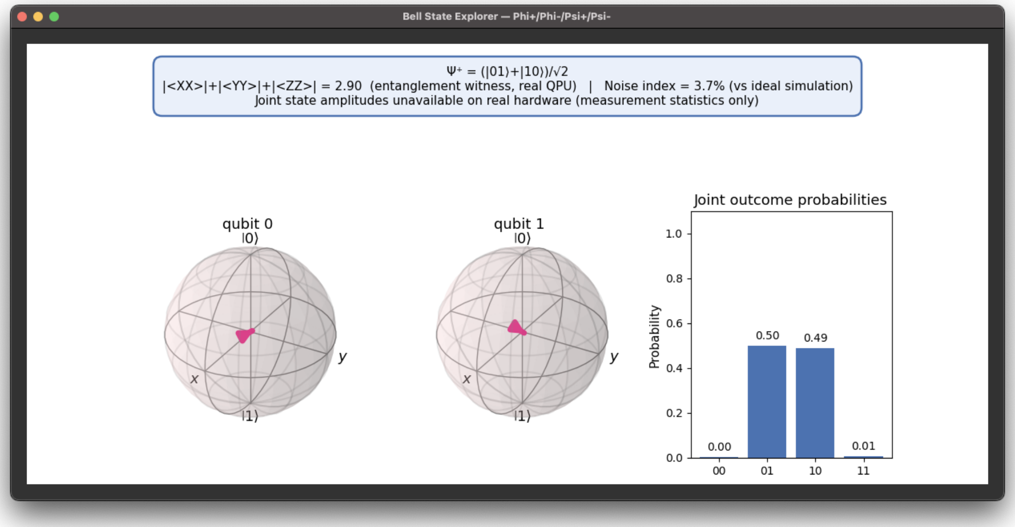
The measured witness is 2.72. The ideal value for this angle is $1 + 2 \sin \vartheta \approx 2.76$, so the hardware reproduces the expected correlations to within about 1.5%. Since the bound for separable states is 1, *the measured value is a clear experimental indication that the two qubits are entangled, and the entanglement survives the noise of the device.* The noise index is 3.8%, corresponding to an average distance of about 0.08 between the measured and ideal Bloch vectors. This is higher than the 1.7% obtained in Experiment 1, which is consistent with the presence of a two-qubit gate. Two-qubit gates are in general the main source of error in current processors.

The two Bloch spheres show vectors of length 0.51 and 0.50, pointing in opposite directions along the vertical axis. Their short length is the expected visual signature of entanglement. *When a qubit is part of an entangled pair, its individual state is mixed, and its vector shrinks inside the sphere.* The ideal length for this angle is $\cos \vartheta \approx 0.47$, so the measured vectors are slightly longer. *This small difference is compatible with the noise, which also modifies the reduced states.* The arrow of qubit 1 is slightly tilted away from the vertical axis. This is a small transverse component that is absent in the ideal case and is attributable to phase errors on the hardware.

The joint probability chart shows $P(01) = 0.71$ and $P(10) = 0.24$, against the ideal values of 0.74 and 0.26. In addition, the outcomes 00 and 11, which are forbidden in the ideal circuit, appear with probabilities 0.01 and 0.04. Together they account for about 5% of the shots, which is a direct measure of the errors of the entangling gate and of the readout. The dominant outcome and the asymmetry between 01 and 10 are the same as in the simulator, where the corresponding figure also showed one strongly dominant outcome and one weaker outcome.

In conclusion, the real processor preserves the main features of the partially entangled state of Experiment 2. These are the reduced Bloch vectors, the asymmetric probabilities and a witness well above the separable bound. The deviations are small and consistent with the noise index. *The most visible effects are the leakage of about 5% of the probability into the forbidden outcomes and the slight lengthening of the vectors.*

4.3.3. Experiment 3: Bell states and density matrix



Ψ⁺ = (|01⟩+|10⟩)/√2
|<XX>|+|<YY>|+|<ZZ>| = 2.90 (entanglement witness, real QPU) | Noise index = 3.7% (vs ideal simulation)
Joint state amplitudes unavailable on real hardware (measurement statistics only)

Figure 4.3.: *Bell state generator with QPU*

Two points before the complete explanation of the image (Figure 4.3):

- Bell state: this run is Ψ^+, while the simulator figure I described earlier showed Ψ^- (see the Figure 3.7 on page 32). The first card's sign and emoji number set the state; it varies between runs. Both states have the same probability support (01 and 10), so the comparison is direct.
- Left panel: on hardware the window shows the two Bloch spheres where the simulator shows the density matrix grid. *The density matrix needs the full state vector, that a real device does not provide.*

The ideal values for Ψ^+ are:

- Witness: 3.0, from $\langle XX \rangle = +1$, $\langle YY \rangle = +1$ and $\langle ZZ \rangle = -1$.
- Bloch vectors: length 0.
- Probabilities: 0.5 for 01 and 0.5 for 10.

Because the ideal vectors have zero length, the noise index here is just the average measured vector length. A 3.7% index corresponds to a length of about 0.07, which matches the tiny arrows in the figure.

The figure (Figure 4.3) shows the result window of Experiment 3 when the circuit is executed on a real IBM quantum processor. The title box reports the Bell state produced for the matched card, $\Psi^+ = (|01\rangle + |10\rangle)/\sqrt{2}$, followed by the entanglement witness and the noise index. The witness is defined as the sum of the absolute values of the three two-qubit correlators ⟨XX⟩, ⟨YY⟩ and ⟨ZZ⟩. *A third line states that the joint amplitudes are not available on real hardware. For this reason the density matrix panel of the simulator version is replaced here by the single-qubit Bloch spheres, which can be reconstructed from measurement statistics alone.*

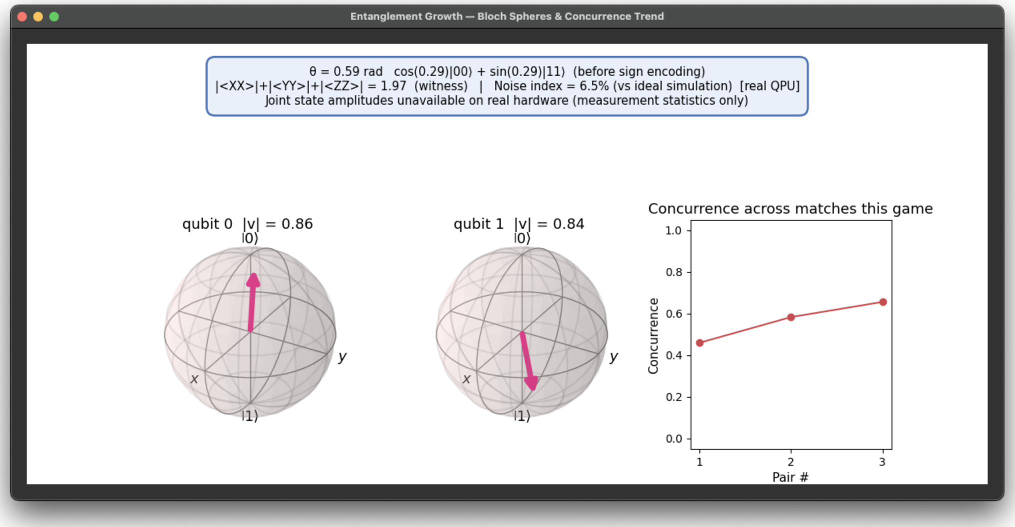
The measured witness is 2.90, against the ideal value of 3.0 for a maximally entangled Bell state. The hardware therefore recovers about 97% of the ideal correlation strength. The separable bound for this quantity is 1, so the measured value lies far above it and certifies the entanglement of the pair. The noise index is 3.7%, a value similar to the 3.8% obtained in Experiment 2. This indicates that the main contribution to the error comes from the entangling gate, and that it does not depend on the particular state being produced.

The two Bloch spheres show very short arrows close to the centre. For a maximally entangled pair the reduced state of each qubit is completely mixed, so the ideal Bloch vector has length zero, as in the simulator. *The residual length, about 0.07 according to the noise index, is the visible effect of the hardware noise.* A pure single-qubit state would be represented by an arrow reaching the surface of the sphere, so the contrast between the two situations makes the presence of entanglement visually evident. The small arrows are also slightly displaced from the centre, which reflects a weak residual polarization of each qubit that is absent in the ideal state.

The joint probability chart shows P(01) = 0.50 and P(10) = 0.49, with P(00) = 0.00 and P(11) = 0.01. The ideal values are 0.5 for each of the two correlated outcomes and 0 for the others. The two outcomes are balanced, as in the simulator figure, and the probability leaking into the forbidden outcomes is only about 1%. The result also illustrates the central property of the state. *Each qubit separately gives a random result, while the pair always gives opposite values.*

In conclusion, the real processor produces the Bell state Ψ^+ with high fidelity. The witness of 2.90 out of 3.0, the nearly maximal mixing of the single-qubit states and the balanced correlated probabilities agree with the simulation. The only visible deviations are small, namely the short residual arrows and the 1% probability in the outcome 11.

4.3.4. Experiment 4: progressive increase of entanglement



Entanglement Growth — Bloch Spheres & Concurrence Trend

θ = 0.59 rad   cos(0.29)|00⟩ + sin(0.29)|11⟩ (before sign encoding)
|<XX>|+|<YY>|+|<ZZ>| = 1.97 (witness) | Noise index = 6.5% (vs ideal simulation) [real QPU]
Joint state amplitudes unavailable on real hardware (measurement statistics only)

Figure 4.4.: *progressive entanglement with QPU*

The figure (Figure 4.4) shows the result window of Experiment 4 on a real IBM quantum processor after the third pair has been found. The title box reports the rotation angle $\vartheta = 0.59$ rad, which in this experiment grows deterministically with the number of matches, and the corresponding state $\cos(0.29)|00\rangle + \sin(0.29)|11\rangle$ before the sign encoding. It also reports the entanglement witness, the noise index, and the statement that joint amplitudes are unavailable on real hardware.

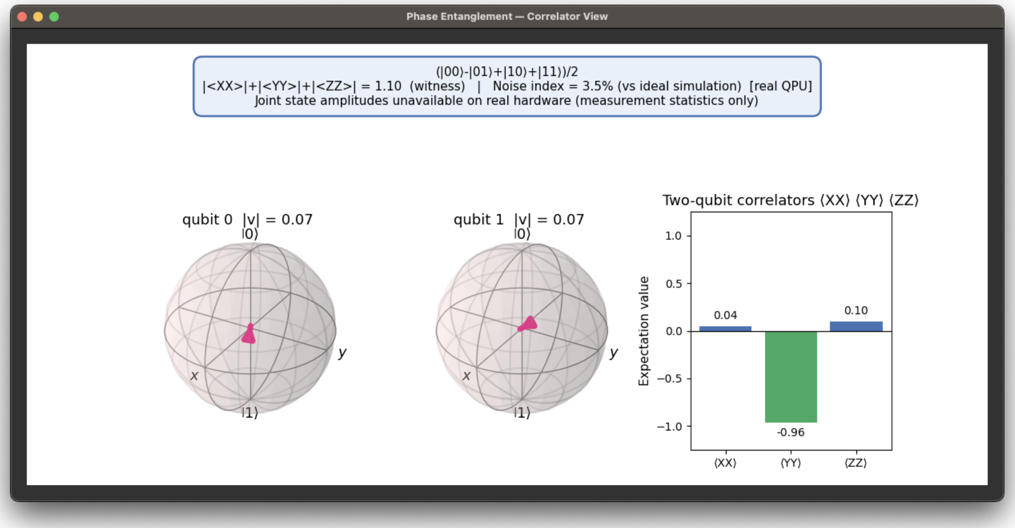
The measured witness is 1.97. The ideal value for this angle is $1 + 2\sin\vartheta \approx 2.11$, so the hardware recovers about 93% of the expected correlation strength. The value is clearly above the separable bound of 1, which confirms that the pair is entangled. The noise index is 6.5%, corresponding to an average distance of about 0.13 between the measured and ideal Bloch vectors. *This is the highest value among the experiments presented so far (1.7% in Experiment 1, 3.8% in Experiment 2, 3.7% in Experiment 3). A possible explanation is a variation in the calibration of the device between the runs, but the data alone do not allow this to be established.*

The two Bloch spheres show vectors of length 0.86 and 0.84, against an ideal length of 0.83. The vectors are still long, since at this stage of the game the entanglement is moderate and each qubit retains a large part of its individual polarization. The two arrows point in nearly opposite directions along the vertical axis, as in the simulation, and are both slightly tilted away from it. The tilt is a transverse component caused by noise, and it is responsible for most of the noise index, since the lengths themselves are close to the ideal ones. Compared with Experiment 2, where $\vartheta = 1.08$ rad gave vectors of length 0.5, the longer vectors here are the visual counterpart of the smaller ϑ.

The graph on the right summarizes the three matches so far. *On hardware the exact concurrence cannot be computed, so the plotted quantity is the witness divided by 3, which acts as an indicator that grows with entanglement.* Its values are about 0.46, 0.58 and 0.66, against the ideal values of 0.46, 0.59 and 0.70 for the same indicator. The curve increases monotonically and follows the ideal trend, with the largest deviation, about 0.04, at the third point. This quantity differs from concurrence, whose ideal values for the same pairs would be 0.20, 0.38 and 0.56. *The indicator never falls below 1/3, even for a product state, so its absolute values should not be compared directly with the simulator curve. The comparison must be made on the trend.*

In conclusion, the hardware reproduces the progressive growth of entanglement along the game. *The witness and the vector lengths agree with the ideal values at the level of a few percent, and the deviation is larger than in the previous experiments, consistently with its higher noise index.*

4.3.5. Experiment 5: maximally entangled state exhibiting


Phase Entanglement — Correlator View

(|00⟩-|01⟩+|10⟩+|11⟩)/2
|<XX>|+|<YY>|+|<ZZ>| = 1.10 (witness) | Noise index = 3.5% (vs ideal simulation) [real QPU]
Joint state amplitudes unavailable on real hardware (measurement statistics only)

Figure 4.5.: *maximally entangled state qubits with QPU*

The figure (Figure on the current page) shows the result window of Experiment 5 when the circuit is executed on a real IBM quantum processor. The title box reports the state produced for the matched card, $(|00\rangle - |01\rangle + |10\rangle + |11\rangle)/2$, followed by the witness and the noise index. A third line states that the joint amplitudes are unavailable on real hardware. In the simulator version, the same state is shown through a density matrix grid in which every cell is populated, with a characteristic pattern of signs. That panel cannot be reconstructed from measurement statistics alone and is therefore replaced on hardware by the single-qubit Bloch spheres.

This experiment differs from the previous ones in an important respect. *The state is entangled only through its phases. The probabilities of the four outcomes are all equal to 0.25, so a measurement in the computational basis gives no sign of correlation.* The entanglement is revealed only by the correlators on the right. The ideal values are $\langle \mathrm{XX} \rangle = 0$, $\langle \mathrm{YY} \rangle = -1$ and $\langle \mathrm{ZZ} \rangle = 0$, and the measured values are 0.04, -0.96 and 0.10. The dominant correlator is therefore recovered with an error of only 4%, and the two correlators that should vanish stay below 0.10. The negative sign of $\langle \mathrm{YY} \rangle$ is reproduced correctly, which shows that the hardware preserves the phase structure of the state and not only the populations.

The witness, defined as the sum of the absolute values of the three correlators, is 1.10. Unlike in Experiments 2 to 4, this quantity cannot be used here to certify entanglement. For this state the ideal value is 1.0, which coincides with the separable bound, because only one correlator is different from zero. The value 1.10 is therefore compatible with the ideal one, and the small excess comes from the residual values of $\langle \mathrm{XX} \rangle$ and $\langle \mathrm{ZZ} \rangle$. *The evidence for entanglement comes instead from the single-qubit vectors.* Their length is 0.07 for both qubits, against an ideal value of exactly zero. For a pure global state, completely mixed single-qubit states imply maximal entanglement. The near-zero length together with the strong $\langle \mathrm{YY} \rangle$ correlation identifies a state that is highly entangled, even though the witness cannot show it. The noise index of 3.5% corresponds to this residual length of about 0.07, and is similar to the values of 3.7% and 3.8% found for Experiments 3 and 2.

In conclusion, the real processor reproduces the phase-entangled state of Experiment 5 with good accuracy. The correlators are within 0.10 of their ideal values, the vectors are almost at the centre of the spheres, and the sign structure is preserved. *The experiment also shows a limit of the witness, which for this class of states stays at the separable bound even though the entanglement is maximal.*

5. Software and architecture

5.1. Overview

For the creation of the software of this thesis I have written the code with two different AI; this has been my first time using AI for the development in Python, because I came from the old school of programming and actually I write mostly scripts in **Bash** and **Ansible** for my daily work.

In any case the experience of programming was good and helped me to realize the game with the minimum effort of coding; but during the development phase, some *"hallucinations"* produced by the artificial intelligence have made me spend some time debugging the code in search of the bugs inserted in the source code.

After the corrections, I have used another artificial intelligence to do a double check of the code and the results; after that I have used the knowledge learned with the quantum computing course of IBM and my experience as programmer gained with big headache and sleepless nights to shape the code of the memory game.

So in this chapter I will face all the development and deployment aspects to make easier for the each reader that would to try make tests on quantum computing with the source code produced with this thesis.

All the code written and the documentation, included also the Lyx files that using to generate this PDF are freely available in the GitHub repository created for this thesis.

To make citations in this document I have create a **DOI** for it; in the following chapters you can find the quotes the code and following the links you can also access and deploy the source code of this project.

5.2. Initialization and deployment

The choose of the language and the components for a new project is always really important for the development and the modification phase.

In this case Python has been a mandatory choice because the Qiskit module is written mostly with it.

To grant portability and a secure environment to make my tests and write new code I have adopted the best practice from the developers standards of today, the module `venv` is an isolated workspace that allows to manage dependencies for different projects separately, preventing conflicts between package versions.

A common way that I use in my works is to create scripts to recreate the original environment in case of issues or simply to *"initialize"* the development folder to re-create everything from scratch. The init script is useful also to deploy the environment on a new computer and also to remember the modules used in Python; quoting a famous latin phrase in the modern era with can say: *"Verba volant, **scripts** manent".*

Here an example of Bash script that I have used to *"populate"* the environment with the software stack used in this project:

bash:

```
pipx install uv
rm -rf .venv
uv venv --python 3.9 .venv
source .venv/bin/activate
uv pip install --upgrade pip setuptools wheel
uv pip install pyqt6 pyqt6-tools notebook
uv pip install qiskit-aer matplotlib
uv pip install qiskit_ibm_runtime
export PATH="$PWD/.venv/bin:$PWD/.venv/lib/python3.9/site-packages/
   qt6_applications/Qt/bin/:$PATH"
alias designer='designer␣-platform␣xcb'
chmod +x ./src/exp1_no_entanglement_random_qubits.py
```

[28, file "init.sh" from GitHub repository]

In the syntax that I have used for init script I have omitted the modules version used but it is required for professional and production environment.

The correct sequence of steps to deploy and after run the scripts on the IBM Quantum Platform can be found in the script named "How-To-Deploy-Schrodinger-Memory.txt".

5.3. Software architecture

The application is organized in four layers, each with a single responsibility: configuration, quantum backend, visualization, and game and graphical interface. The code is written in Python and amounts to about 2,500 lines, of which roughly three quarters belong to the game layer. The remaining quarter is shared by all experiments. This division was chosen so that the five experiments could differ from one another only in their quantum content, while everything else stayed unchanged.

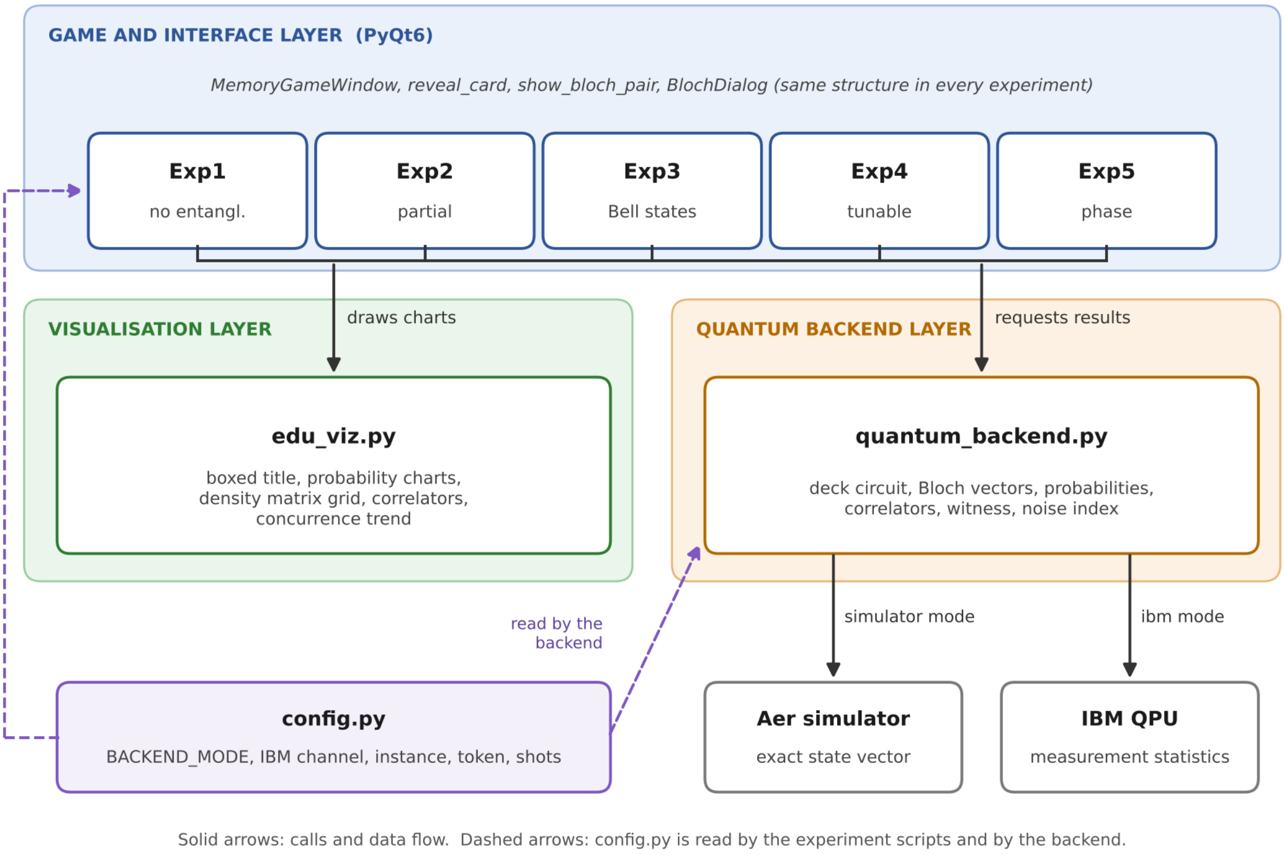

GAME AND INTERFACE LAYER (PyQt6)
MemoryGameWindow, reveal_card, show_bloch_pair, BlochDialog (same structure in every experiment)
Exp1
no entangl.
Exp2
partial
Exp3
Bell states
Exp4
tunable
Exp5
phase
draws charts
requests results
VISUALISATION LAYER
edu_viz.py
boxed title, probability charts, density matrix grid, correlators, concurrence trend
QUANTUM BACKEND LAYER
quantum_backend.py
deck circuit, Bloch vectors, probabilities, correlators, witness, noise index
read by the backend
simulator mode
ibm mode
config.py
BACKEND_MODE, IBM channel, instance, token, shots
Aer simulator
exact state vector
IBM QPU
measurement statistics
Solid arrows: calls and data flow. Dashed arrows: config.py is read by the experiment scripts and by the backend.

Figure 5.1.: *software layers*

5.3.1. Configuration layer

A single file, `config.py`, selects how the program runs. The main parameter chooses between the local simulator and a real IBM quantum processor. Further parameters define the IBM channel, the account instance, the access token, the name of the device and the number of shots per circuit. Every other module reads this file, so switching from simulation to hardware requires editing only one place, and no experiment contains code that is specific to a backend.

5.3.2. Quantum backend layer

The module `quantum_backend.py` hides the differences between the simulator and the real device behind a common set of functions. These functions run the circuit that generates the deck, compute the Bloch vectors of the two qubits, return the joint outcome probabilities and the correlators $\langle XX \rangle$, $\langle YY \rangle$ and $\langle ZZ \rangle$, and compute the entanglement witness and the noise index. On the simulator the quantities are computed exactly from the state vector. On the real device they are estimated from measurements, using sampling for the probabilities and expectation values of Pauli operators for the vectors and correlators. Quantities that exist only in simulation, such as the state vector and the exact concurrence, are not available on hardware, and the backend returns an empty value in their place. The graphical code recognizes this case and adapts the content of the window. The module also contains small helper functions, among them a random-angle generator based on quantum measurements, which assigns the entanglement angle used in Experiment 2.

5.3.3. Visualization layer

The module `edu_viz.py` collects the educational charts as independent functions, each drawing one chart on a given `Matplotlib` axis. They produce the boxed title, the joint probability chart, the per-qubit probability chart, the density matrix grid, the correlator chart and the concurrence trend. These functions know nothing about the game or about the graphical toolkit, so each experiment can freely choose which of them to combine in its result window.

5.3.4. Game and interface layer

The experiments has been written as a separate script, derived from the original memory game, and all five have the same structure. A function converts the measured bits of the deck circuit into cards. The main window, built with `PyQt6`, manages the grid of sixteen buttons, the flipping of cards, the matching rule and the hiding of non-matching pairs. When a pair is matched, a single method builds the two-qubit circuit of the experiment, requests the results from the backend, composes the charts and shows them in a dialog that embeds the `Matplotlib` figure in a Qt window. This method is the only part that changes substantially from one experiment to another. The graphical interface is identical in all of them, in accordance with the design constraint of modifying only the circuit and its initialization.

5.3.4.1. Flow of a match

When the player turns two cards, the game checks the matching rule. If the cards match, the experiment circuit is built, including the conditional gates that depend on the sign of the first card and on the parity of the emoji index. The circuit is then executed by the backend, on the simulator or on the quantum processor according to the configuration. A new circuit is used for this purpose, because the measurement that generated the deck has already destroyed the state of the original qubits. The results are finally drawn as charts in the result window.

5.3.4.2. Design rationale

Four choices follow from this structure; the separation of concerns keeps *configuration*, *quantum computation*, *drawing* and *game logic* in distinct modules, so that a change in one does not affect the others. The abstraction of the backend gives simulator and hardware the same interface, which makes the direct comparison of the two modes, presented in the results chapter, possible with identical code. The minimal modification of the original script keeps the interface familiar to the player and limits the differences between experiments to the quantum circuit. Finally, the graceful handling of unavailable quantities allows the same window code to show exact values in simulation and estimated values on hardware.

5.3.5. Software design choices

The application is written in Python 3 and uses four main libraries. Qiskit and its Aer simulator provide the quantum circuits and the local simulation. The package `qiskit-ibm-runtime` gives access to real IBM processors. `PyQt6` provides the graphical interface, and `Matplotlib` draws the educational charts, which are embedded in Qt windows. Python was chosen because all of these libraries are available for it and because it is the language of the Qiskit ecosystem. Each listing is identified by file, function and line range, and all files are in the `src` folder of the repository.

5.3.5.1. Encoding of a card

A card is the outcome of a measurement on four qubits. The function `bits_to_card` converts the four bits into a sign and an index, and the index selects one of eight emojis (Listing 1).

Listing 1: *Conversion of a four-bit measurement into a card (*`src/exp3_all_bell_states.py`*, function* ***`bits_to_card`****, lines 51 to 55).*

python:

```python
def bits_to_card(bitstring: str) -> str:
sign = '+' if bitstring[-1] == '0' else '-'
idx = int(bitstring[-4:-1], 2) # three bits -> 0-7
return f"{sign}{EMOJIS[idx]}"
```

[28, file "exp3_all_bell_states.py" from GitHub repository]

The least significant bit gives the sign and the other three bits give the emoji index, so the sixteen possible outcomes are the eight emojis in two signs. Two cards match when they show the same emoji with opposite signs, which gives eight pairs on the board. The method `generate_deck` (line 150 of the same file) runs the four-qubit circuit with one measurement per run, keeps only the cards not yet obtained, and stops when all sixteen are collected. The order on the board is then shuffled classically with random.shuffle.

One interface for simulator and hardware. The central design decision is that the experiments never ask which device is running. The module `quantum_backend.py` offers one function per task, and each function reads the setting `BACKEND_MODE` from the configuration file and dispatches internally (Listing 2).

Listing 2: *Backend selection (*`src/config.py`*, lines 27 and 32, and* `src/quantum_backend.py`*, function* ***`run_deck_circuit`****, lines 98 to 121).*

python:

```
BACKEND_MODE = "ibm"                          # "simulator" or "ibm"
IBM_CHANNEL  = "ibm_quantum_platform"
```

[28, file "config.py" from GitHub repository]

python:

```
def run_deck_circuit(qc):
    if config.BACKEND_MODE == "ibm":
        backend = _get_ibm_backend()
        tqc = transpile(qc, backend)
        job = Sampler(backend).run([tqc], shots=1)
        ...
    else:
        sim = AerSimulator(method='statevector')
        sim.set_options(shots=1)
        ...
```

[28, file "quantum_backend.py" from GitHub repository]

With this structure, a run on a real processor needs a change in only one line of the configuration file, which is what made the comparison between simulation and hardware in the results chapter possible with identical code. In the tagged release the setting selects the real processor, and the value `"simulator"` runs the game locally with no account. The access token is not stored in the repository, and the setting `IBM_TOKEN = None` makes the program use an account saved locally by the user.

Mapping of the observables on hardware. On a real processor the transpiler places the two qubits of the experiment on physical qubits of its choice, in a device that has more than one hundred qubits. For this reason each observable is remapped with the layout produced by the transpiler (Listing 3). Without this step the expectation values would refer to different qubits from those of the circuit, and the Bloch vectors would be meaningless. All six single-qubit observables are sent in a single job, which also limits the use of the available quantum time.

Listing 3: *Mapping of the observables onto the physical qubits (*`src/quantum_backend.py`*, function* `get_bloch_vectors`*, lines 143 to 155).*

python:

```python
obs = []
for q in range(n_qubits):
    for p in "XYZ":
        label = ["I"] * n_qubits
        label[q] = p
        obs.append(SparsePauliOp("".join(reversed(label))).apply_layout(
            tqc.layout))
job = estimator.run([(tqc, obs)])
evs = [float(v) for v in job.result()[0].data.evs]
vectors = [evs[3 * q: 3 * q + 3] for q in range(n_qubits)]
```

[28, file "quantum_backend.py" from GitHub repository]

What changes between experiments. The five experiments share the same window, the same matching rule and the same deck generation, and they differ in the method `show_bloch_pair`, which builds the circuit for the matched pair. In Experiment 3, for instance, the Bell state is selected by two properties of the match (Listing 4).

Listing 4: *Circuit of Experiment 3 (*`src/exp3_all_bell_states.py`*, method* `show_bloch_pair`*, lines 269 to 276).*

python:

```python
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)                          # |Phi+>
if first_lbl[0] == '−':
    qc.z(0)                          # phase flip
emoji_idx = EMOJIS.index(first_lbl[1:])
if emoji_idx % 2 == 1:
    qc.x(1)                          # bit flip
```

[28, file "exp3_all_bell_states.py" from GitHub repository]

Here the sign of the first card controls the phase gate, and the parity of the emoji index controls the bit flip. The parity of the emoji index is used because, under the matching rule, a pair always contains one card of each sign, so the signs alone would allow only two of the four Bell states.

5.3.5.2. Adaptation to the limits of hardware

A real processor does not return the state vector, so the backend functions return an empty value for quantities that exist only in simulation. The window code tests for it and shows different content: exact amplitudes and concurrence in simulation, and measured quantities (witness, correlators, noise index) on hardware.

5.3.5.3. Numerical robustness

One function deals with a numerical problem of the library. The concurrence function of Qiskit can return a value that is not a number for almost unentangled states, because rounding errors make the argument of a square root slightly negative. All concurrence calls go through the wrapper `safe_concurrence` (Listing 5), which clips the result to the interval from 0 to 1.

Listing 5: *Protection against invalid concurrence values (*`src/quantum_backend.py`*, function* `safe_concurrence`*, lines 332 to 339).*

python:

```python
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", message="invalid␣value␣encountered␣
        in␣sqrt")
    c = concurrence(Statevector(statevec))
c = float(c)
if math.isnan(c):
    c = 0.0
return max(0.0, min(1.0, c))
```

[28, file "quantum_backend.py" from GitHub repository]

5.3.5.4. Embedding of the charts in the interface

The result window is a Qt dialog that contains a `Matplotlib` canvas (Listing 6). The charts are therefore built as ordinary `Matplotlib` figures by the module `edu_viz.py`, and they can also be saved as images for the thesis.

Listing 6: *Dialog that embeds a figure (*`src/exp2_bell_concurrence.py`*, class* `BlochDialog`*, lines 73 to 82).*

python:

```
class BlochDialog(QtWidgets.QDialog):
    def __init__(self, fig, parent=None):
        super().__init__(parent)
        self.setMinimumSize(1000, 560)
        layout = QtWidgets.QVBoxLayout(self)
        canvas = FigureCanvas(fig)
        layout.addWidget(canvas)
```

[28, file "exp2_bell_concurrence.py" from GitHub repository]

6. Conclusion

6.1. Overview

This thesis started from a personal question: *can a person with no background in quantum computing learn its basic ideas by building and studying a small interactive game?* I came from the world GNU Linux system administration, and quantum computing was a subject that I did not know at all. To answer the question I created a memory game in which every card is produced by quantum measurements, and I added five experiments, each showing a different kind of entanglement between two qubits. The same code was run on a simulator and on a real IBM quantum processor. This chapter summarizes what the experiments showed, the practical limits met when working with real hardware, the role of artificial intelligence in the project, and what the work has meant for my learning.

6.2. From simulation to real hardware

The five experiments covered the main situations of a two-qubit system.

- **Experiment 1:** *prepared two independent qubits with no entanglement.*
- **Experiment 2:** *produced a pair with a partial entanglement that depends on a random angle.*
- **Experiment 3:** *produced the four Bell states, selected by the sign and the emoji of the matched cards.*
- **Experiment 4:** *increased the entanglement along the game.*
- **Experiment 5:** *produced a state whose entanglement lies only in the phases.*

In the simulator all quantities are exact. On the processor they are estimated from measurements, so the comparison shows how far real hardware stays from the ideal behavior. Table 6.1 collects the entanglement witness, the sum of the absolute values of the correlators $\langle \mathrm{XX} \rangle$, $\langle \mathrm{YY} \rangle$ and $\langle \mathrm{ZZ} \rangle$, together with the noise index of each run.

Experiment	Ideal witness	Measured witness	Noise index
1. Independent qubits	1.00	1.07	1.7%
2. Partial entanglement	2.76	2.72	3.8%
3. Bell state Ψ^+	3.00	2.90	3.7%
4. Third match	2.11	1.97	6.5%
5. Phase entanglement	1.00	1.10	3.5%

Table 6.1. Entanglement witness and noise index of the hardware runs, compared with the ideal values.

Three conclusions follow from these results. The first is that *the real processor reproduces the qualitative behavior of every experiment.* The correlated outcomes of the Bell state, the reduced length of the Bloch vectors in entangled pairs, the growth of the witness along Experiment 4 and the sign of the dominant correlator in Experiment 5 are all visible on the hardware, with small deviations from the simulation. The second is that *the noise is moderate.* The noise index stays between 1.7% and 6.5%, and the lowest value belongs to the experiment without an entangling gate, which agrees with the fact that two-qubit gates are the main source of error in present devices. The third conclusion concerns a *limit of the instrument.* In Experiment 5 the witness remains at the separable bound of 1, even though the state is maximally entangled, so a single indicator cannot replace the full set of correlators and vectors. The entanglement of that state is recognized only by combining the strong $\langle \mathrm{YY} \rangle$ correlation with Bloch vectors close to zero length.

These results must be read with caution. Each figure comes from a single run on a single device, so the values give the order of magnitude of the noise and not a statistical measurement. *A different run, or the same device on another day, could give different numbers.*

6.3. Practical limits of access to quantum hardware

Working on a real quantum processor requires planning that a simulator does not. A new IBM account gives 10 minutes of free quantum computing time, and all my tests needed about 15 minutes. The jobs also wait in a queue, whose duration is not under the control of the user, and the free account is limited in the resources it can use, as described in Chapter 4 on page 39. In the game the quantum time is consumed quickly, because a deck is generated by repeating a four-qubit circuit with one measurement per run and discarding the cards already obtained. About 54 runs are needed on average to complete the sixteen cards, and each run is a separate job. Each matched pair then requires further jobs for the experiment circuit and its observables.

For a person who wants to explore the quantum world, the practical consequence is a way of working. *Development and debugging should be done on the simulator*, which is instant and exact, and the hardware should be kept for a small number of final runs whose results are worth the time. The design adopted here follows this idea, since a single line of the configuration file switches between the two backends, and the same code runs unchanged. The limitation also suggests a direction for improvement, namely a deck generation that uses fewer jobs.

6.4. The use of artificial intelligence

Artificial intelligence was used throughout the project, for the code, for the explanation of concepts, for the circuits and for the drafting of text. This was my first experience of using it for development in Python. It was a powerful instrument, and it let me spend more time reading articles, following the quantum course and fixing bugs. It also showed that its output cannot be accepted without checking, because *it can contain errors that look convincing.* As described in Chapter 5 on page 49, some *hallucinations* inserted bugs in the code that I had to find, and for this reason I used a second AI to double check the code and the results before relying on them. Some of the errors that appeared during this work are listed in Table 6.2.

Error	**Correction**
A label stated that the witness should be about zero for independent qubits	The ideal value for the antipodal pair is about 1, which is also the separable bound
A trend graph on hardware was labeled concurrence but plotted the witness divided by three	The quantity is described in the text as a witness-based indicator
The Bell state was described as chosen by the game, and the comparison with the simulator as direct	The text names the real selection rule and limits the comparison to probabilities and witness
The first hardware backend did not map the observables onto the physical qubits chosen by the transpiler	The released code applies the layout of the transpiler to every observable

Table 6.2. Examples of errors in the output of the AI assistants and their corrections.

The common point of these cases is that the error was not a failure to produce an answer, *but a plausible answer that did not match the physics, the code or the data.* The method that worked was to treat the assistants as fast collaborators whose statements are verified: asking for the reasoning, reading the code, comparing with the ideal values, using a second tool as a reviewer, and asking again when a result looked wrong. This verification required enough understanding of the subject to notice a problem, which is a reason why the learning process and the use of AI cannot be separated. *The conclusion for modern projects is that AI is an effective tool when it is monitored, and that the responsibility for the correctness of the work remains with the author.*

6.5. What this work has meant for learning

At the beginning of this project I did not know quantum computing at all. This thesis follows the path that took me from that starting point to a game that generates its cards from quantum measurements, to five experiments on entanglement and to their execution on a real processor. Several choices made the path shorter. Building a game gave me a concrete object to reason about, instead of an abstract formalism. The visual outputs, Bloch spheres, density matrices and correlator charts, allowed me to see what each state looks like before I could describe it with mathematics. The simulator and the hardware used identical code, so the difference between the ideal and the real behavior became an observation and not just a statement in a textbook. The IBM course and the Qiskit documentation gave the theoretical base, and the AI assistants acted as tutors, provided that their explanations were checked.

The resulting material is also reusable. The code, the circuits and the descriptions are archived in a public repository with a DOI, so a reader with the same starting point can follow the same steps, repeat the experiments and change them. I consider this the main contribution of the work: a method and a set of tools that lower the entry barrier to a subject that appears difficult at first, and not only a collection of results.

6.6. Limitations and future work

The work has several limits! The hardware results come from single runs on one device, and each experiment uses only two qubits. The entanglement witness is not sufficient for every state, as Experiment 5 showed. The generation of the deck is expensive on real hardware. The educational effect of the game has been described, but it has not been measured with learners.

Several developments are possible, the deck generation could be redesigned to use fewer jobs. State tomography could be added on hardware, to reconstruct the density matrix at the cost of more quantum time. Error mitigation techniques could be applied to reduce the effect of noise, and the experiments could be repeated on different devices and on different days to obtain statistics. Larger entangled states with three or more qubits would extend the game to richer phenomena. A study with students would test whether the approach really shortens the learning path, which is the claim that this thesis can only support from a single experience.

6.7. Final remarks

A small game was enough to bring a complex subject within reach, and the same code, run on a simulator and on a real processor, showed both the beauty of entanglement and the imperfection of present machines. The work also showed that the new tools, from cloud quantum computers to AI assistants, are valuable when they are used with curiosity and with care. *This is the main message that I wish to leave to the reader!*

Acknowledgments

Thanks to my family, that shown me the path.

Thanks to my love, that never disappear.

Thanks to friends, that gave me the help to rise up.

Thanks to colleagues, for the wisdom.

Thanks to all the people that believe in a better future.

Thanks to G-d to be here.

Thanks to the *Prof. Luca Potì*, that helped me to complete the experiments.

Thanks to you that have spent your time to read this thesis!

A. Appendix

A.1. Overview

All the material produced for this thesis is freely available in a public repository on GitHub, in line with the principles of free software described in the abstract. The repository contains the source code of the experiments, the files used to write this document, the pictures used in the chapters and the screenshots obtained from the real quantum processor. The version cited in this thesis is v0.1.3, archived with the DOI [https://doi.org/10.5281/zenodo.23773774].

A.2. Links

Resource	**Content**	**Link**
GitHub repository	The whole project: Python source code, initialization script and deployment instructions	Shrodinger-Memory/tree/v0.1.3
LyX folder (`doc`)	The LyX files of all the chapters and of the bibliography, used to generate this PDF	Shrodinger-Memory/tree/v0.1.3/doc
Image folder (`img`)	The images used in the chapters, such as circuits, charts and illustrations	Shrodinger-Memory/tree/v0.1.3/img
Picture folder (`pic`)	The screenshots captured during the runs on the IBM quantum processor, one folder for each experiment	Shrodinger-Memory/tree/v0.1.3/pic

Table A.1. Online resources of the thesis (version v0.1.3).

The most recent state of the project, which may be ahead of the cited version, is available at https://github.com/david-pixel-wise/Shrodinger-Memory/.

A.3. License

The code and the documents in the repository are released under a permissive free software license (the MIT license), whose text is in the file LICENSE of the repository. This allows anyone to run, study, modify and redistribute the material, in accordance with the four freedoms described in the abstract.

A.4. How to run the experiments

The steps below summarize how to repeat the experiments. The detailed instructions for Linux are in the file `txt/How-To-Deploy-Schrodinger-Memory.txt` of the repository.

1. Clone the repository from the link in Table A.1.
2. Run the script `init.sh`, which creates a virtual environment with Python 3.9 and installs the required packages (`PyQt6`, `Qiskit`, `Qiskit Aer`, `Matplotlib` and the module `qiskit-ibm-runtime`). Then activate the environment with the command `source .venv/bin/activate`.
3. Open the file `src/config.py` and set the value of `BACKEND_MODE`. The value `"simulator"` runs the game locally and needs no account. The value `"ibm"` runs it on a real IBM processor, and it is the value used in the tagged release.
4. For the real processor, create an account on the IBM Quantum Platform and save the credentials once on your computer, with the channel set to `ibm_quantum_platform`. The access token is never stored in the repository.
5. Run one of the five experiment files according with the correct file name, for example `python src/exp1_no_entanglement_random_qubits.py`.

A.5. Images and file names

The Table A2 give the correspondences between the images and file names.

Figure ID	File name
Figure 1.1 on page 7	HeadTail_Plot1.png
Figure 1.2 on page 7	HeadTail_Plot2.png
Figure 2.1 on page 16	deck_visualization.png
Figure 2.2 on page 18	deck_generation_circuit.png
Figure 2.3 on page 19	exp1_ch2_circuit.png
Figure 2.4 on page 20	exp2_ch2_circuit.png
Figure 2.5 on page 21	exp3_ch2_circuit.png
Figure 2.6 on page 22	exp4_ch2_circuit.png
Figure 2.7 on page 23	exp5_ch2_circuit.png
Figure 3.1 on page 26	ch3_complex_numbers_as_vectors.png
Figure 3.2 on page 27	ch3_vector_length_and_probability.png
Figure 3.3 on page 28	ch3_bloch_sphere_illustration.png
Figure 3.4 on page 29	exp1_ch3_deck1.png
Figure 3.5 on page 30	exp1_ch3_visualization1.png
Figure 3.6 on page 31	exp2_ch3_visualization1.png
Figure 3.7 on page 32	exp3_ch3_visualization1.png
Figure 3.8 on page 34	exp4_ch3_visualization1.png
Figure 3.9 on page 34	exp4_ch3_visualization5.png
Figure 3.10 on page 36	exp5_ch3_visualization1.png
Figure 4.1 on page 41	exp1_ibm_visualization1.png
Figure 4.2 on page 42	exp2_ibm_visualization1.png
Figure 4.3 on page 44	exp3_ibm_visualization1.png
Figure 4.4 on page 46	exp4_ibm_visualization1.png
Figure 4.5 on page 47	exp5_ibm_visualization1.png
Figure 5.1 on page 51	architecture_diagram.png

Table A.2. Figures of the thesis and the corresponding image files.

A.6. Dates of the hardware runs

The screenshots in the picture folder were captured on the dates in Table A.3.

Experiment	Folder	Date
1	`pic/exp1_ibm`	8 October 2026
2	`pic/exp2_ibm`	10 October 2026 (one image on 14 October 2026)
3	`pic/exp3_ibm`	19 October 2026
4	`pic/exp4_ibm`	19 October 2026
5	`pic/exp5_ibm`	19 October 2026

Table A.3. Dates of the screenshots of the runs on the IBM quantum processor.

Bibliography

[1] E. Schrödinger, “Die gegenwärtige Situation in der Quantenmechanik.”
DOI: https://doi.org/10.1007/BF01491891, 1935.

[2] GNU and F. S. Foundation, “Free software.”
URL: https://www.gnu.org/philosophy/free-sw.html
(visited 16-07-2026).

[3] Wikipedia, “Color Depth.”
URL: https://en.wikipedia.org/wiki/Color_depth
(visited 22-07-2026), 2020.

[4] Wikipedia, “Color Depth, Illustration at 2 bits.”
URL: https://upload.wikimedia.org/wikipedia/commons/5/56/2_bit.png
(visited 22-07-2026), 2017.

[5] Wikipedia, “Color Depth, Illustration at 8 bits.”
URL: https://upload.wikimedia.org/wikipedia/commons/f/ff/8_bit.png
(visited 22-07-2026), 2017.

[6] Wikipedia, “Color Depth, Illustration at 24 bits.”
URL: https://upload.wikimedia.org/wikipedia/commons/9/9f/24_bit.png
(visited 22-07-2026), 2017.

[7] Intel, “Intel©64 and IA-32 Architectures Software Developer’s Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4.”
URL: https://cdrdv2.intel.com/v1/dl/getContent/671200
(visited 16-07-2026), 2026.

[8] D. Adams, *Hitchhiker’s guide to the galaxy.* Pocket Books, 1984.
ISBN: 9780671527211.

[9] N. Bansal and M. Sviridenko, “The Santa Claus problem.”
DOI: http://dx.doi.org/10.1145/1132516.1132522, may 2006.

[10] R. Mariescu-Istodor and P. Fränti, “Solving the Large-Scale TSP Problem in 1h: Santa Claus Challenge 2020.”
DOI: http://dx.doi.org/10.3389/frobt.2021.689908, 2021.

[11] J. Schneider, I. Smalley, and IBM, “QPU (Quantum Processing Unit).”
URL: https://www.ibm.com/think/topics/qpu
(visited 15-08-2026), april 2026.

[12] M. Ivezic, “Schrödinger’s Wave Equation.”
URL: https://postquantum.com/quantum-computing/schrodingers-equation/
(visited 27-07-2026), 2019.

[13] T. Bunn, “Animations of solutions to the Schrodinger equation.”
URL: https://facultystaff.richmond.edu/%7Eebunn/phys205/waves/
(visited 27-07-2026), 2026.

[14] A. R. Douglas, “I don’t understand quantum physics.”
URL:
https://www.southampton.ac.uk/~doug/quantum_physics/quantum_physics.pdf
(visited 25-07-2026), 2018.

[15] Wikipedia, “Transporter.”
URL: https://en.wikipedia.org/wiki/Transporter_(Star_Trek)
(visited 27-07-2026), 2026.

[16] Wikipedia, “The no-communication theorem.”
URL: https://en.wikipedia.org/wiki/No-communication_theorem
(visited 26-10-2026), 2026.

[17] E. Davis, “Teleportation Physics Study.”
URL: https://sgp.fas.org/eprint/teleport.pdf
(visited 27-07-2026), august 2004.

[18] IBM, “Understanding Quantum Information & Computation With John Watrous.”
URL: https://youtu.be/42OiBzfdE2o
(visited 25-06-2026), 2022.

[19] IBM, “Multiple Qubits and Entangled States.”
URL: https://github.com/Qiskit/textbook/tree/main/notebooks/ch-gates
(visited 27-07-2026), 2023.

[20] J. S. Bell, “On the Einstein Podolsky Rosen paradox.”
URL: https://cds.cern.ch/record/111654/files/vol1p195-200_001.pdf
(visited 27-07-2026), november 1964.

[21] Wikipedia, “John Stewart Bell.”
URL: https://en.wikipedia.org/wiki/John_Stewart_Bell
(visited 27-07-2026), 2026.

[22] M. Sisodia, “Comparison the performance of five-qubit IBM quantum computers in terms of Bell states preparation: M. Sisodia.”
DOI: https://doi.org/10.1007/s11128-020-02712-7, 2020.

[23] Ravensburger, “Memory Ravensburger.”
URL: https://www.ravensburger.it/it-IT/ravensburger/products/games/games-for-kids/classic-large-memory-20889
(visited 16-08-2026), 2026.

[24] Wikipedia, “The Solovay–Kitaev theorem.”
URL: https://en.wikipedia.org/wiki/Solovay%E2%80%93Kitaev_theorem
(visited 17-08-2026), 2026.

[25] C. Q. Choi, “Diamond-Based Quantum Accelerator Puts Qubits in a Server Rack.”
URL: https://spectrum.ieee.org/nitrogen-vacancy-diamond-quantum-computer-accelerator-qubits-server-rack
(visited 26-10-2026).

[26] E. Angelini, “IBM Launches Its Most Advanced Quantum Computers, Fueling New Scientific Value and Progress towards Quantum Advantage.”
URL: https://newsroom.ibm.com/2024-11-13-ibm-launches-its-most-advanced-quantum-computers,-fueling-new-scientific-value-and-progress-towards-quantum-advantage
(visited 23-10-2026), november 2024.

[27] Wikipedia, “The no-cloning theorem.”
URL: https://en.wikipedia.org/wiki/No-cloning_theorem
(visited 26-10-2026), 2026.

[28] D. J. Giovannini, “david-pixel-wise/Shrodinger-Memory: v0.1.3.”
URL: https://github.com/david-pixel-wise/Shrodinger-Memory/
(visited 23-10-2026), 2026.

Nomenclature

AI	Artificial intelligence is the capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making.
Algorithm	In mathematics and computer science, an algorithm is a finite sequence of mathematically rigorous instructions, typically used to solve a class of specific problems or to perform a computation.
Ansible	Ansible is a suite of software tools that enables infrastructure as code. It is open-source and the suite includes software provisioning, configuration management, and application deployment functionality.
Bash	Bash (short for "Bourne Again SHell") is an interactive command interpreter and command language developed for Unix-like operating systems. Supported by the Free Software Foundation and created in 1989 by Brian Fox for the GNU Project, it is designed as a completely free software alternative for the Bourne shell, sh, and other proprietary Unix shells.
Bit	The bit is the most basic unit of information in computing and digital communication. The name is a portmanteau of binary digit.
Boolean logic	Boolean logic is a type of algebra that deals with true and false values, using operators like AND, OR, and NOT to combine these values. It is fundamental in computer science and digital circuit design, allowing for reasoning about binary conditions.
Browser	A web browser, often abbreviated as browser, is an application for accessing websites.
Bug	A software bug is a defect in computer software.
CNOT gate	The CNOT gate, or Controlled-NOT gate, is a two-qubit quantum logic gate that flips the state of the target qubit only if the control qubit is in the state entangled states and is a fundamental component in quantum computing.
CPU	Central processing unit, is the primary processor in a computer.
Dirac notation	Dirac notation, also known as bra-ket notation, is a mathematical framework used in quantum mechanics to describe quantum states and operations.

DOI	They are widely used to identify academic, professional, and government information, such as journal articles, research reports, data sets, and official publications. A DOI aims to resolve to its target, the information object to which the DOI refers.
Ebit	An ebit is one unit of bipartite entanglement, defined as the amount of entanglement in a maximally entangled two-qubit state, such as a Bell state.
Emojis	The word emoji comes from Japanese e (“picture”) + moji (“character”) and originally meant “pictograph”; the primary function of modern emoji is to fill in emotional cues otherwise missing from typed conversation.
Energize	In Star Trek, "Energize" is the classic command given to activate the ship’s transporter beam, converting matter into an energy pattern for teleportation, or to engage power systems.
Entanglement	Quantum entanglement is the phenomenon in which the quantum state of each particle in a group cannot be described independently of the state of the others, even when the particles are separated by a large distance.
Epoch	Epoch time, also known as Unix time, is defined as the number of seconds that have elapsed since 00:00:00 UTC on January 1, 1970.
Erwin Schrödinger	Erwin Rudolf Josef Alexander Schrödinger, sometimes anglicised as Schroedinger or Schrodinger, was an Austrian theoretical physicist who developed fundamental results in quantum theory.
FOSS	Free and Open Source Software.
Free software	Free software or libre software sometimes known as freedom-respecting software is computer software distributed under free software licenses that allow users to run the software for any purpose as well as to study, change, and distribute it and any adapted versions.
Git	Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency. Git is lightning fast and has a huge ecosystem of GUIs, hosting services, and command-line tools.
GPU	Graphics processing unit.
GUI	Graphical user interface.
Hadamard gate	The Hadamard gate is a single-qubit quantum gate that transforms the basis states $\|0\rangle$ and $\|1\rangle$ into superposition states.
Jupyter	The Jupyter Notebook is a web-based interactive computing platform. The notebook combines live code, equations, narrative text, visualizations, interactive dashboards and other media.

Logic gate	A logic gate is a device that performs a Boolean function, a logical operation performed on one or more binary inputs that produces a single binary output.
LSP	Large scale problem.
NPU	Neural processing unit.
Open source software	Open-source software is computer software with publicly available source code, so people can use, study, modify, and distribute it under an open license.
Pauli gates	The Pauli gates are the three Pauli matrices and act on a single qubit. The Pauli X, Y and Z equate, respectively, to a rotation around the x, y and z axes of the Bloch sphere by radians.
Python	Python is a high-level, general-purpose programming language that emphasizes code readability, simplicity, and ease-of-writing with the use of significant indentation, an extensive ("batteries-included") standard library, and garbage collection. Python supports multiple programming paradigms but with an emphasis on object-oriented programming and dynamic typing.
Qiskit	Software development kit, Python based, developed by IBM as high-performance software stack for quantum computing.
QPU	Quantum processing unit.
Qt	Qt is a cross-platform application development framework for creating graphical user interfaces as well as cross-platform applications that run on various software and hardware platforms such as Linux, Windows, macOS, Android or embedded systems with little or no change in the underlying codebase while still being a native application with native capabilities and speed.
Quantization	Quantization is the process of mapping input values from a large set (often a continuous set) to output values in a (countable) smaller set, often with a finite number of elements.
Quantum computing	A quantum computer is a computer that represents and processes information using quantum states. Quantum computations exploit phenomena such as superposition, interference, and entanglement.
Qubit	A qubit can be physically realized with a two-state (or two-level) quantum-mechanical system, one of the simplest quantum systems displaying the peculiarity of quantum mechanics.
Sci-fi	Sci-fi is short for science fiction, a storytelling genre based on futuristic science, space travel, and advanced technology.
Scripts	In Linux, scripts are files containing sequences of commands that the shell executes automatically in order, often starting with a shebang line to specify the interpreter.

SDK	Software development kit.
Star Trek	Star Trek is an American science fiction media franchise created by Gene Roddenberry and owned by Paramount, which began with the series of the same name and became a worldwide pop culture phenomenon.
Superposition	Superposition is a fundamental principle in physics, particularly in quantum mechanics, stating that a system can exist in multiple states at once until it is measured.
Transporter	A transporter is a fictional teleportation machine used in the Star Trek universe.
TSP	The Traveling Salesman Problem (TSP) is a classic algorithmic challenge in computer science and operations research.
Witness	In quantum information theory, an entanglement witness is a functional which distinguishes a specific entangled state from separable ones.
x86_64	x86-64 (also known as AMD64, x86_64, x64 and, rarely, Intel 64) is a 64-bit extension of the x86 instruction set. It was announced in 1999 and first available in the AMD Opteron family in 2003.